

ICS 2025 期末复习

张毅 (yi_zhang@mail.ustc.edu.cn)

期末考试

- 考试时间：1月17日 08:30~10:30
- 开卷考试
- 题型设置
 - Part A: 多项选择题
 - Part B: 简答题
 - Part C: LC-3 架构
 - Part D: 数据结构
 - Part E: 综合程序题

第 2 章 位和数据类型

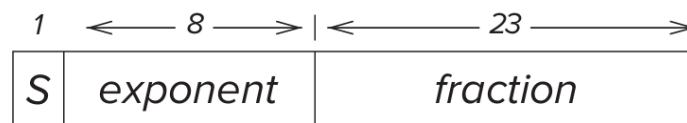
- 整数类型

- 无符号整数：所有位都参与值的表示。
- 有符号整数：补码表示
 - 非负数的最高位权重是 2^k ，负数的最高位权重是 -2^k 。
 - 表示范围。
 - 正负数间补码的转换。
 - 判断补码算术的溢出。
 - 符号扩展、零扩展。
- 二进制、十进制和十六进制之间的转换
- 逻辑运算：与、或、非、异或
 - `ADD(NOT(ADD(x5A, xFF)), x5A)`

第 2 章 位和数据类型

- 浮点类型

- 符号位、指数位（带偏置）和尾数位（隐含的前导 1）
- 规格数（指数 1 ~ 254）
- 非规格数（指数为 0）、无穷大和 NaN（指数为 255）
- 非 32 位浮点数：按照题目给定的位数计算
- 特殊性质的数：关键是理解浮点数表示规则
 - 表示范围、最大的非规格数、最小的规格数、最小的浮点正数.....
- 判断：*A 32-bit floating-point number can precisely represent any integer value that can be represented by a 32-bit two's complement integer. (×)*

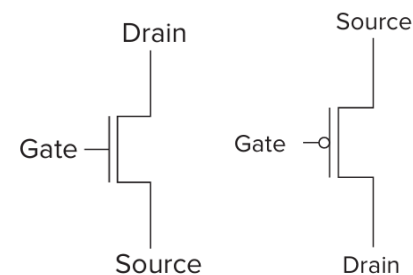


- 字符和字符串：ASCII 码表

第 3 章 数字逻辑

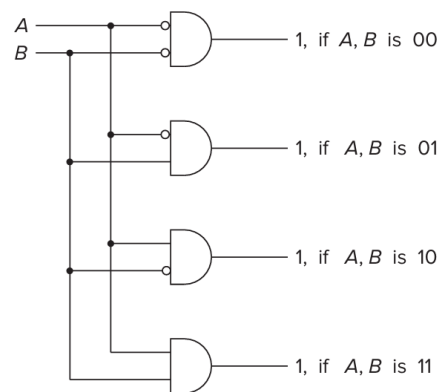
- 逻辑门

- N 型 / P 型 MOS 晶体管的特性
- NOT、OR / NOR、AND / NAND 门，及其真值表
- 逻辑完备

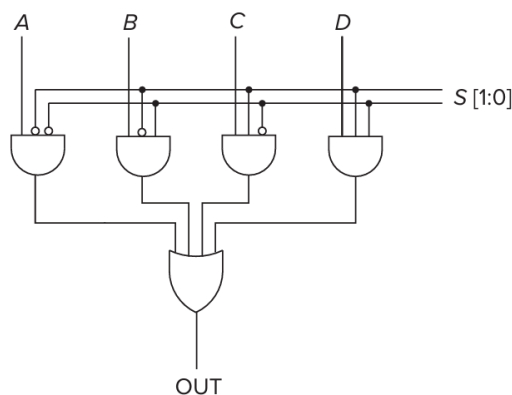


- 组合逻辑电路

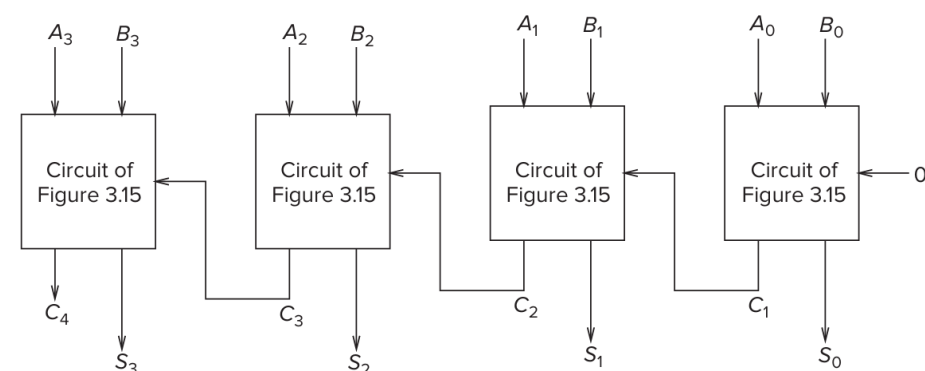
- 译码器



多路选择器



全加器



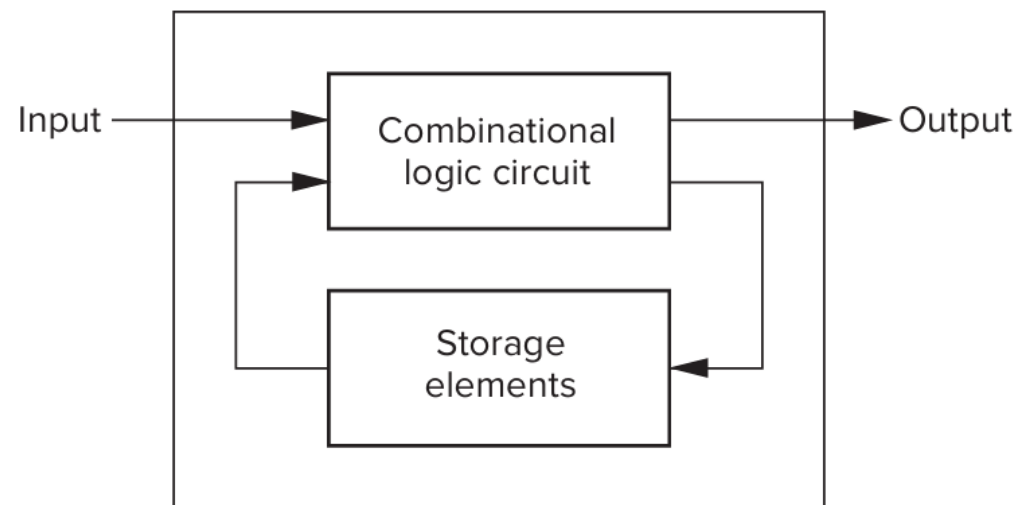
第 3 章 数字逻辑

- 内存

- Address space: 内存中拥有的内存位置的数量（决定了地址的位数）
- Addressability: 每个内存位置存储的位数（决定了字长）
 - *For a byte-addressable memory, an ASCII character occupies one memory location. (√)*
- 锁存器: R-S Latch / Gated D Latch / flip-flops / register

- 时序电路和有限状态机

- 组合逻辑: 实现状态转移
- 存储单元: 保存当前状态
- 时钟: 驱动状态转移



第 4 章 冯·诺依曼模型

- 组成部分

- 处理单元

- ALU: ADD、AND、NOT、PASSA
 - 寄存器文件: 8 个寄存器

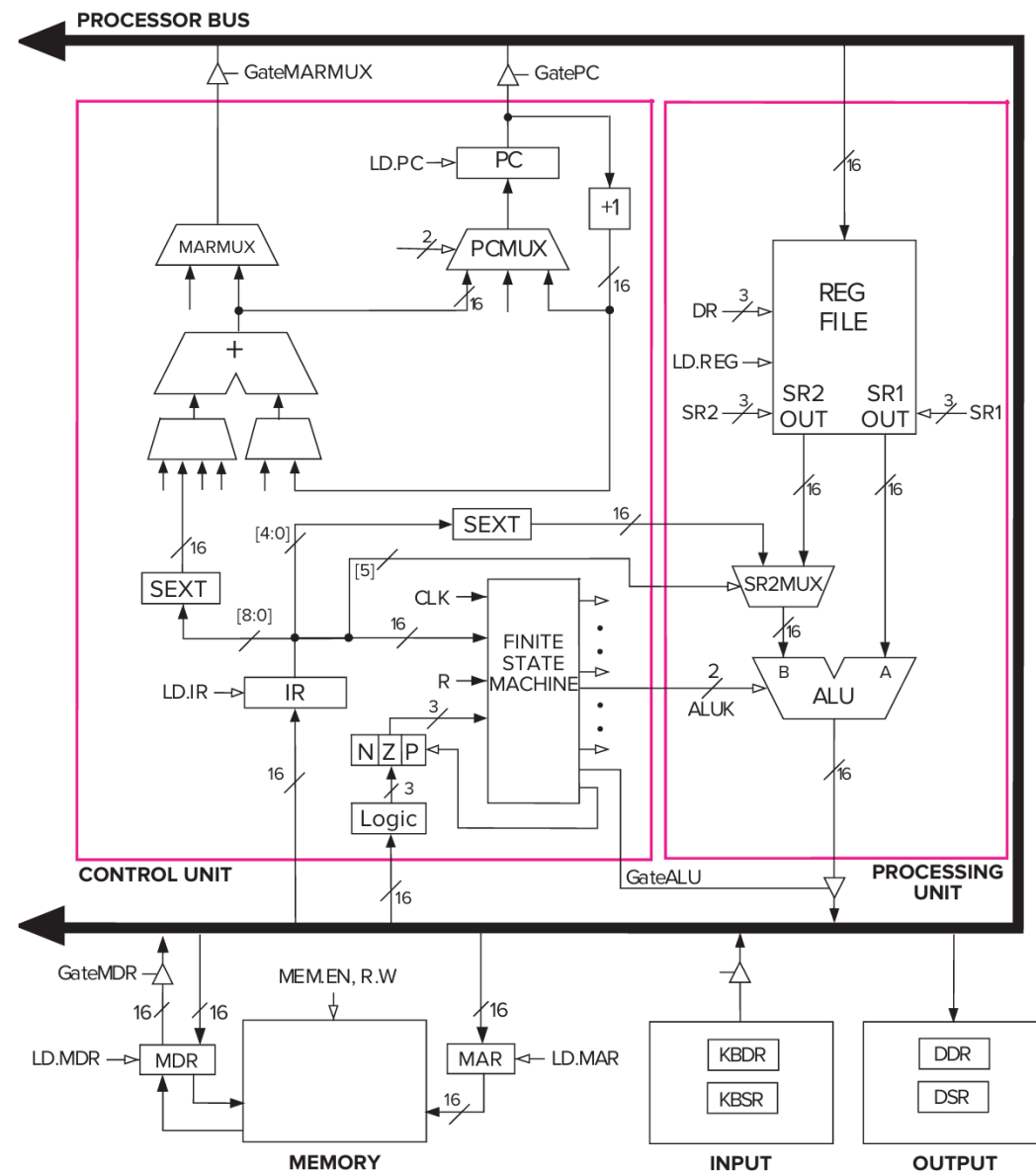
- 控制单元

- IR: 当前指令
 - PC: 下一条指令的地址
 - 驱动指令执行的状态机和时钟

- 内存

- MAR、MDR
 - Address space、Addressability

- 输入、输出

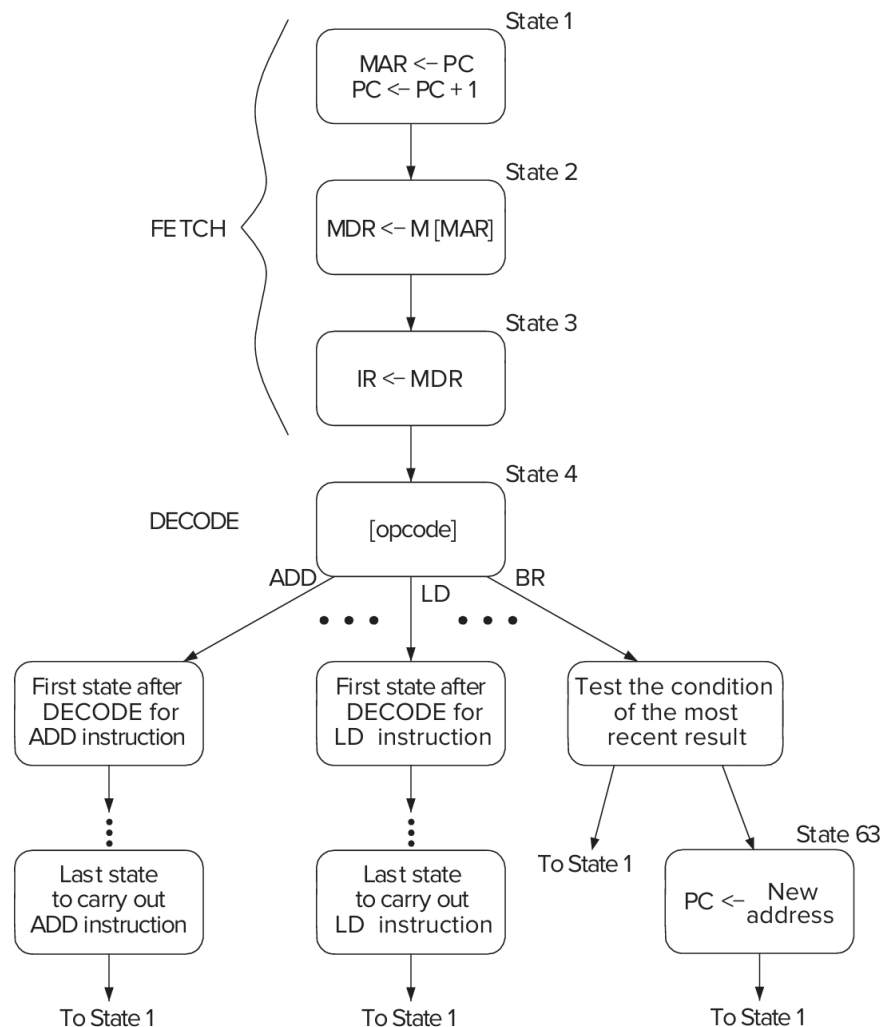


第 4 章 冯·诺依曼模型

- 指令处理

- Fetch
- Decode
- Evaluate Address
- Fetch Operands
- Execute
- Store Result

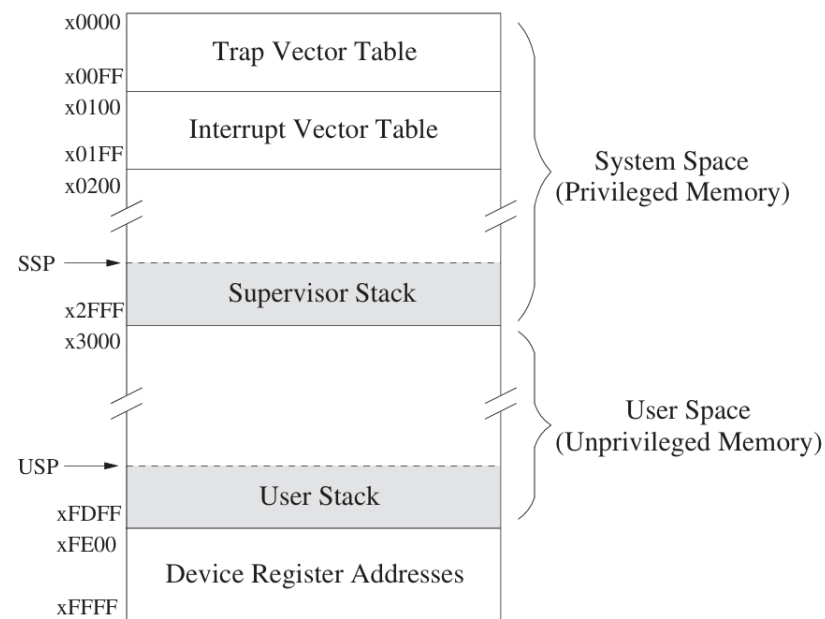
- 考试时以附录 C 的状态机和数据通路为准



第 5 章 / 附录 A LC-3

- 地址空间

- 16 位地址 (2^{16} 个地址)
- 每个地址存储 16 位数据
- 特权内存: x0000 ~ x2FFF
- 内存映射 IO: xFE00 ~ xFFFF, 使用访存指令访问硬件寄存器
- Supervisor Stack: 使用 SSP 管理
- User Stack: 使用 USP 管理

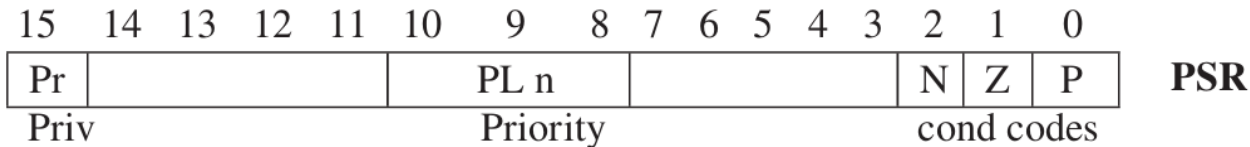


第 5 章 / 附录 A LC-3

- 寄存器
 - 通用寄存器 GPR: R0 ~ R7
 - 设备寄存器

Table A.1 Device Register Assignments		
Address	I/O Register Name	I/O Register Function
xFE00	Keyboard status register (KBSR)	The ready bit (bit [15]) indicates if the keyboard has received a new character.
xFE02	Keyboard data register (KBDR)	Bits [7:0] contain the last character typed on the keyboard.
xFE04	Display status register (DSR)	The ready bit (bit [15]) indicates if the display device is ready to receive another character to print on the screen.
xFE06	Display data register (DDR)	A character written in bits [7:0] will be displayed on the screen.
xFFFC	Processor Status Register (PSR)	Contains privilege mode, priority level and condition codes of the currently executing process.
xFFFE	Machine control register (MCR)	Bit [15] is the clock enable bit. When cleared, instruction processing stops.

- Processor Status Register (PSR)
 - 考题里可能会有对 PSR 的位操作



第 5 章 / 附录 A LC-3

- 寄存器
 - 条件码寄存器（1 bit）
 - N、Z、P 的含义
 - 哪些指令会修改条件码：LD、LDI、LDR、AND、ADD、NOT
 - PC 计数器：下一条指令的地址
- 指令集
 - 全部指令的含义、格式和用法（附录 A.2）
 - 容易被忽略的 LEA 指令
 - 设计新指令
 - 指令编码

第 5 章 / 附录 A LC-3

- 内存寻址模式

- PC 相对寻址 (LD 和 ST 指令)

- PC + 立即数作为目标地址
 - PC 在 Fetch 阶段已经自增, 所以以下一条指令的地址为基地址
 - 立即数在参与运算前要做符号扩展

- 间接寻址 (LDI 和 STI 指令)

- 先用 PC 相对寻址得到一个地址
 - 再从该地址中读取目标地址

- 寄存器相对寻址 (LDR 和 STR 指令)

- 寄存器 + 立即数作为目标地址
 - 立即数在参与运算前要做符号扩展

第 5 章 / 附录 A LC-3

- 控制指令

- 条件跳转 (BRxx 指令)

- 跳转目标相对于 PC
 - 实现循环：通常涉及向后跳转

- 无条件跳转 (BR 和 JMP)

- JMP 指令跳转到目标寄存器存储的地址上

- TRAP 和 RTI

- 预定义的中断向量的含义 (表 A.3)
 - RTI 用于从中断 / 陷阱 / 异常服务程序中返回

- 过程调用

- JSR 和 JSRR 指令：跳转到目标过程并将返回地址存储到 R7，注意二者表达目标地址的形式
 - RET (JMP R7)：从函数返回

第 7 章 汇编

- 汇编器
 - 指令文本 -> 二进制程序
- LC-3 汇编指令格式
 - 标签 操作码 操作数 注释
 - 立即数前缀：x (hex)、# (dec)、b (bin)
- 指令编码（附录 A.2）
 - 以标签表示目标地址，注意该地址在指令中的编码方式
 - AND 和 ADD 指令有 2 种格式
 - 立即数的有效位数（某个地址的最大范围）

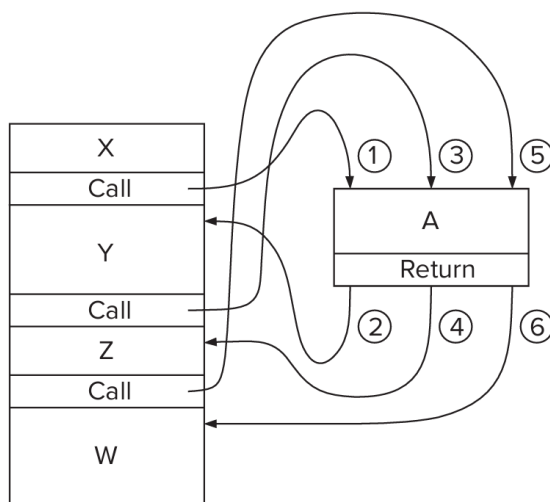
第 7 章 汇编

- 伪指令

- 指导汇编器行为，不产生硬件指令
- .ORIG: 指定首条指令的地址
- .FILL: 使用给定的值填充下一个位置
- .BLKW: 分配指定数量的内存位置
- .STRINGZ: 使用给定字符串初始化后续的内存位置
 - 注意字符串末尾包含隐式添加的 0
- .END: 标注汇编程序的末尾
 - 注意该指令不能用于终止运行的程序

第 8 章 数据结构

- 过程调用
 - 调用 / 返回机制
 - 被调用者（callee）和调用者（caller）



第 8 章 数据结构

- 过程调用

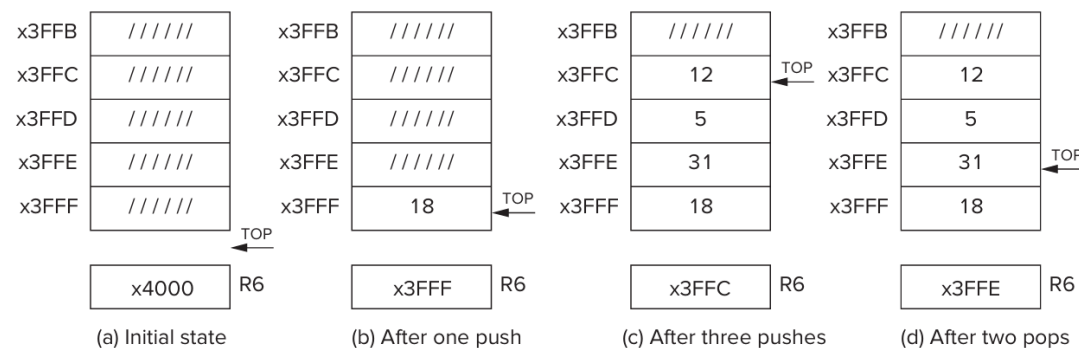
- JSR: 以 PC 相对寻址的方式确定目标函数的入口地址
- JSRR: 以寄存器作为目标地址
- 返回地址 (PC) 保存在 R7 寄存器
 - 和 BR 的区别在于 BR 不会保存返回地址
- 通过 JMP R7 (RET) 指令从过程中返回

- 寄存器的保存和恢复

- 被调用者保存: 过程中用到的、且调用者不明确的寄存器需要由被调用者保存
- 调用者保存: 函数文档明确说明了会被修改的寄存器需要由调用者保存 (如 R7)

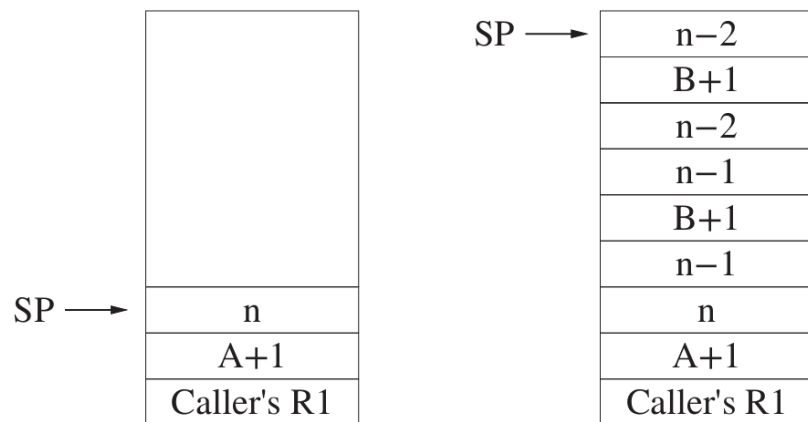
第 8 章 数据结构

- 栈（stack）
 - 后进先出（LIFO）
 - Push：将元素压到栈顶；Pop：将栈顶元素弹出
 - 使用 LC-3 或 C 实现
 - Underflow：试图从空栈中弹出元素
 - Overflow：试图向满栈中压入元素
- 实现方式
 - 连续的内存空间
 - 指向栈顶元素的指针（考试时以题目描述为准）



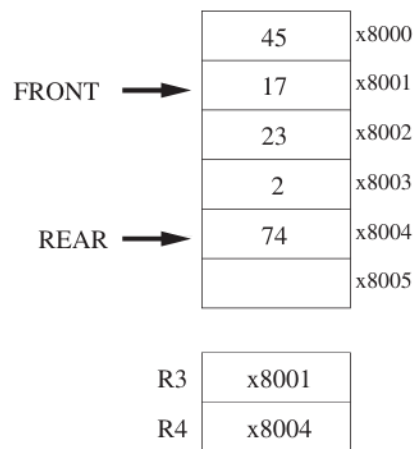
第 8 章 数据结构

- 递归
 - 函数自身调用自身
 - 明确函数的输入和输出寄存器
 - 在递归函数中保存寄存器
 - 使用栈保存寄存器和返回地址



第 8 章 数据结构

- 队列（queue）
 - 先进先出（FIFO）
 - Insert：将元素放到队尾；Remove：将队首元素移除
 - Underflow：试图从空队列中移除元素
 - Overflow：试图向满队列中插入元素
 - Wrap-around：指针移动到队尾时，回绕到队首
 - 实现方式
 - 连续的内存空间
 - 指向队首和队尾的指针（考试时以题目描述为准）



第 9 章 I/O

- 程序权限

- Supervisor mode: 允许执行所有指令和访问任意内存
- User mode: 不能执行 RTI 指令和访问特权内存
- Privilege mode exception: User mode 的程序尝试执行 RTI 指令
- Access Control Violation (ACV) exception: User mode 的程序尝试访问特权内存
- PSR[15] 为 1, 则处于 User mode。

- 优先级

- LC-3 包含 8 个优先级, 位于 PSR[10:8]。
- 高优先级的程序和外部设备能中断低优先级的程序

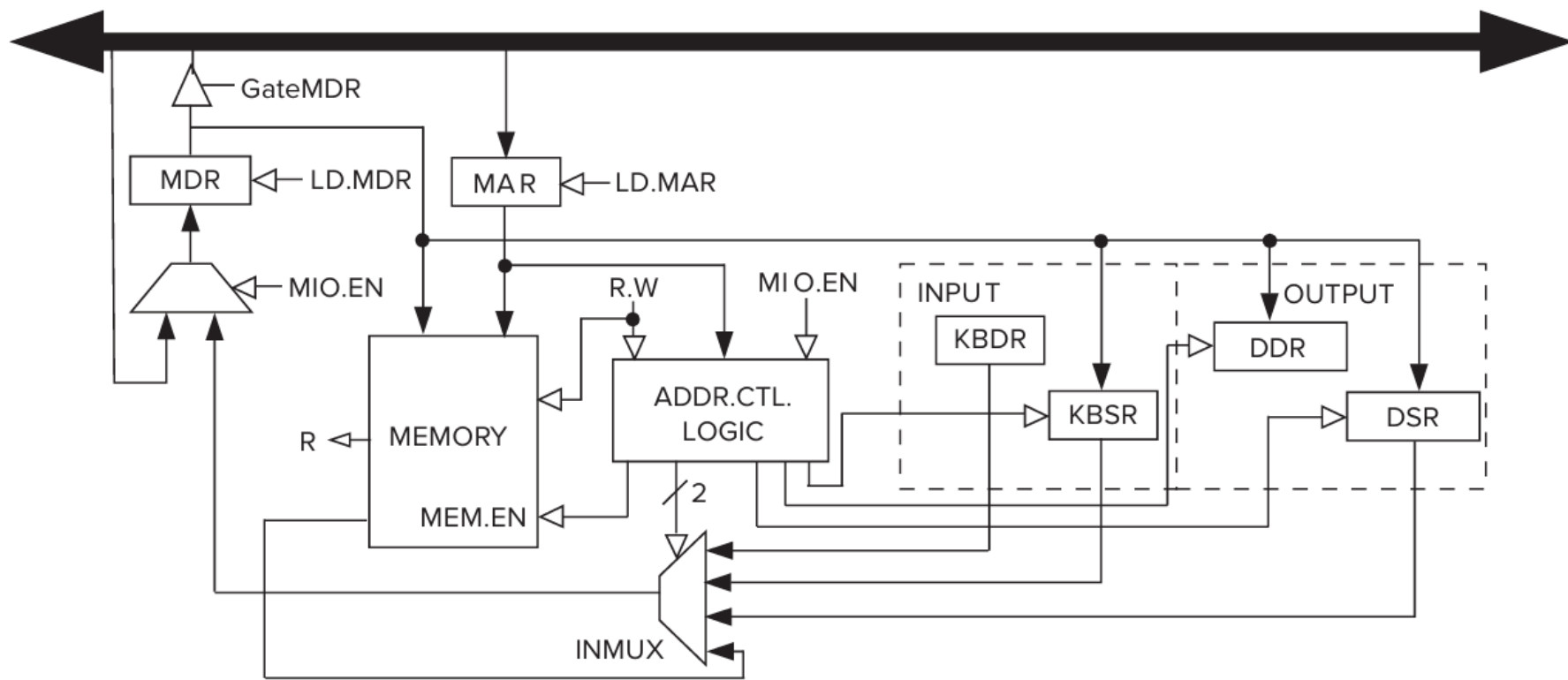
- 内存布局

第 9 章 I/O

- Memory-mapped I/O
 - 将设备寄存器映射到 xFE00 ~ xFFFF 地址空间中
 - 使用常规访存指令访问设备寄存器
- Asynchronous I/O
 - 外部设备的频率相比于处理器要低得多
 - 处理器和外部设备的通信必须是异步的：不能锁步执行
 - 需要某种同步机制等待 IO 设备完成：标志设备状态的寄存器

第 9 章 I/O

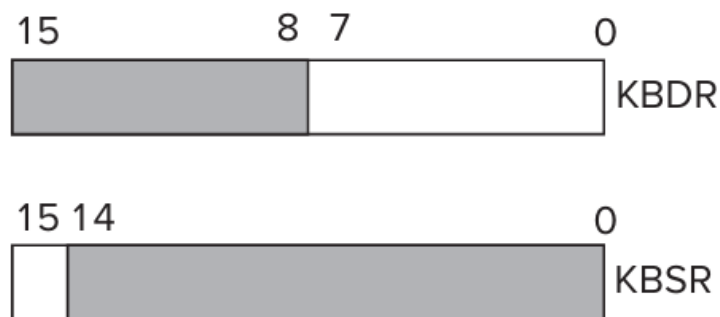
- 内存映射 IO 的实现



第 9 章 I/O

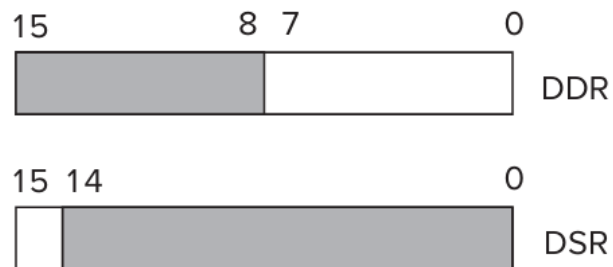
- 从键盘输入

- KBDR (xFE00) : 用户输入字符的 ASCII 码
- KBSR (xFE02) : 表示用户是否已经输入了字符
- 输入字符时, 将 KBSR[15] 设为 1
- 读取 KBDR 时, 自动清零 KBSR[15]



第 9 章 I/O

- 输出到屏幕
 - DDR (xFE06) : 目标字符的 ASCII 码
 - DSR (xFE04) : 表示屏幕是否就绪、
 - 处理完当前字符后, 将 DSR[15] 设为 1



第 9 章 I/O

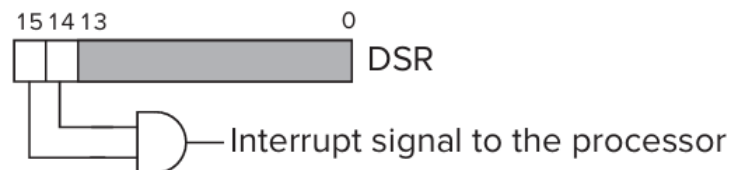
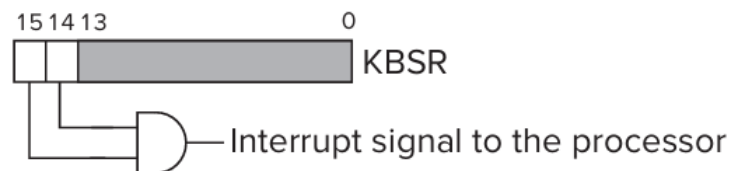
- Polling
 - 由处理器控制和外部设备的交互
 - 在循环中轮询状态寄存器，直到状态可用
 - 处理器一直在空转：浪费计算资源
- Interrupt-Driven I/O
 - 由 IO 设备控制和处理器的交互
 - 当外部设备可用时，请求处理器中断当前程序，执行 IO 处理
 - 允许处理器在 IO 设备执行期间执行其它任务

第 9 章 I/O

- 中断机制

- 发起时机

- 当 KBSR 或 DSR 就绪时，设备发起中断
 - Interrupt enable (IE) bit: 由处理器控制，表示 IO 设备是否有权利发起中断
 - 用于关闭中断



- 请求的优先级

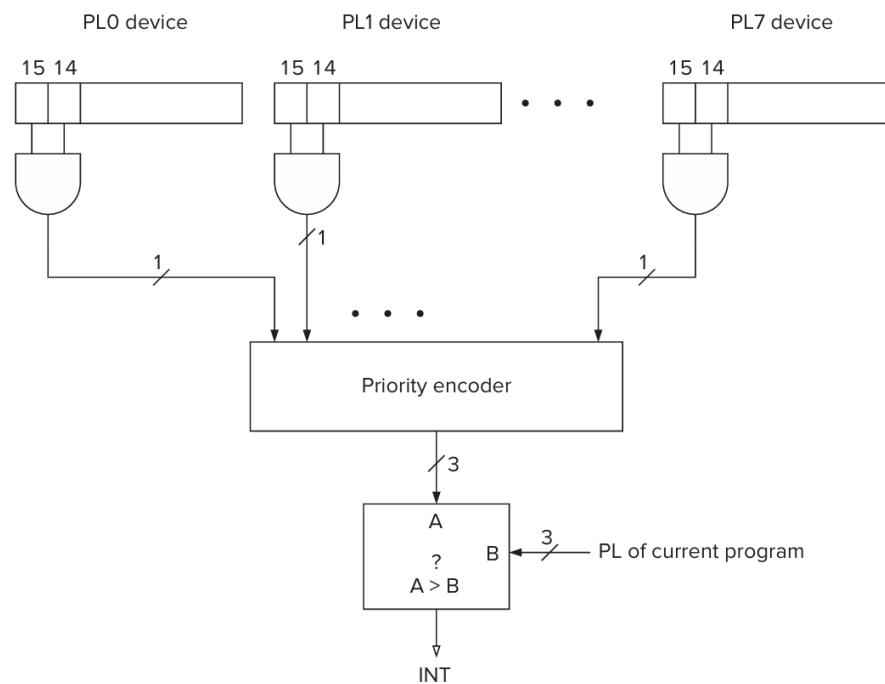
- 该设备的优先级必须比当前正在执行的程序的优先级高
 - 该设备的优先级必须比其它中断请求的优先级高

第 9 章 I/O

- 中断机制

- INT 信号的处理

- INT 信号：中断处理器的当前程序，转入中断服务程序
- 中断可以在任何时刻发起，但只有在当前指令的最后一个周期才受理



第 9 章 I/O

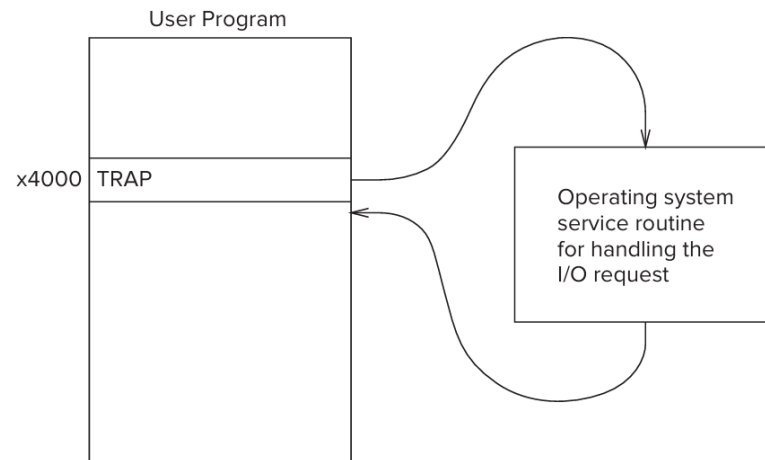
- Trap 指令（附录 A.3.1）

- 执行流程（Execute 阶段）

- 将 PSR 和 PC 保存到 supervisor stack，此时 PC 指向下一条指令（即恢复位置）
 - 将程序调整至特权模式：PSR[15] = 0
 - 如果原程序在用户模式，将 R6 保存到 Saved_USP，并把 Saved_SSP 设为 R6
 - 根据陷阱 / 中断向量，确定服务程序的入口地址
 - 跳转到服务程序执行

- 从服务程序返回（RTI 指令）

- 从 supervisor stack 的栈顶弹出两个元素到 PSR 和 PC 中
 - 根据程序权限调整 R6 的值



第 9 章 I/O

- Trap 指令（附录 A.3.1）

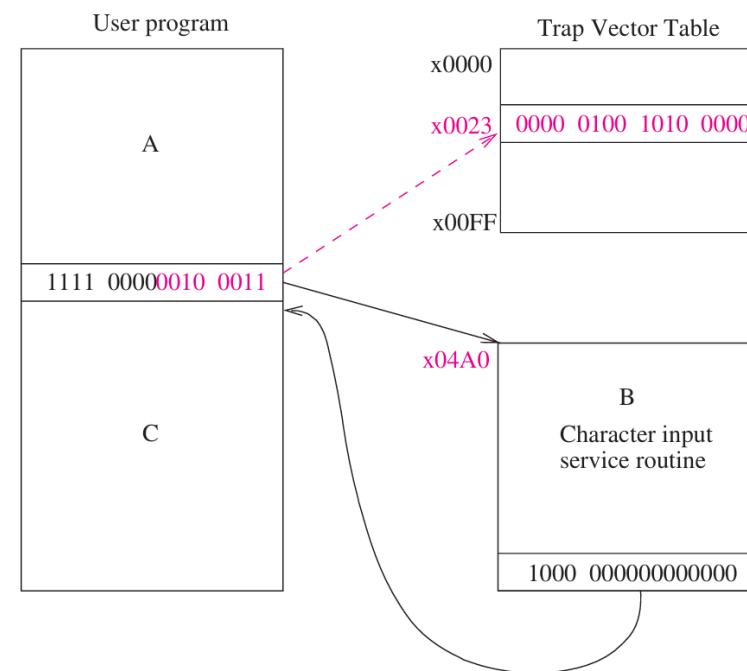
- Trap Service Routines

Table A.3 Trap Service Routines

Trap Vector	Assembler Name	Description
x20	GETC	Read a single character from the keyboard. The character is not echoed onto the console. Its ASCII code is copied into R0. The high eight bits of R0 are cleared.
x21	OUT	Write a character in R0[7:0] to the console display.
x22	PUTS	Write a string of ASCII characters to the console display. The characters are contained in consecutive memory locations, one character per memory location, starting with the address specified in R0. Writing terminates with the occurrence of x0000 in a memory location.
x23	IN	Print a prompt on the screen and read a single character from the keyboard. The character is echoed onto the console monitor, and its ASCII code is copied into R0. The high eight bits of R0 are cleared.
x24	PUTSP	Write a string of ASCII characters to the console. The characters are contained in consecutive memory locations, two characters per memory location, starting with the address specified in R0. The ASCII code contained in bits [7:0] of a memory location is written to the console first. Then the ASCII code contained in bits [15:8] of that memory location is written to the console. (A character string consisting of an odd number of characters to be written will have x00 in bits [15:8] of the memory location containing the last character to be written.) Writing terminates with the occurrence of x0000 in a memory location.
x25	HALT	Halt execution and print a message on the console.

- 入口地址

- Trap Vector Table: 地址 x0000 ~ x00FF
- Interrupt Vector Table: 地址 x0100 ~ x1FF
 - x0100 ~ x017F: 异常服务的入口



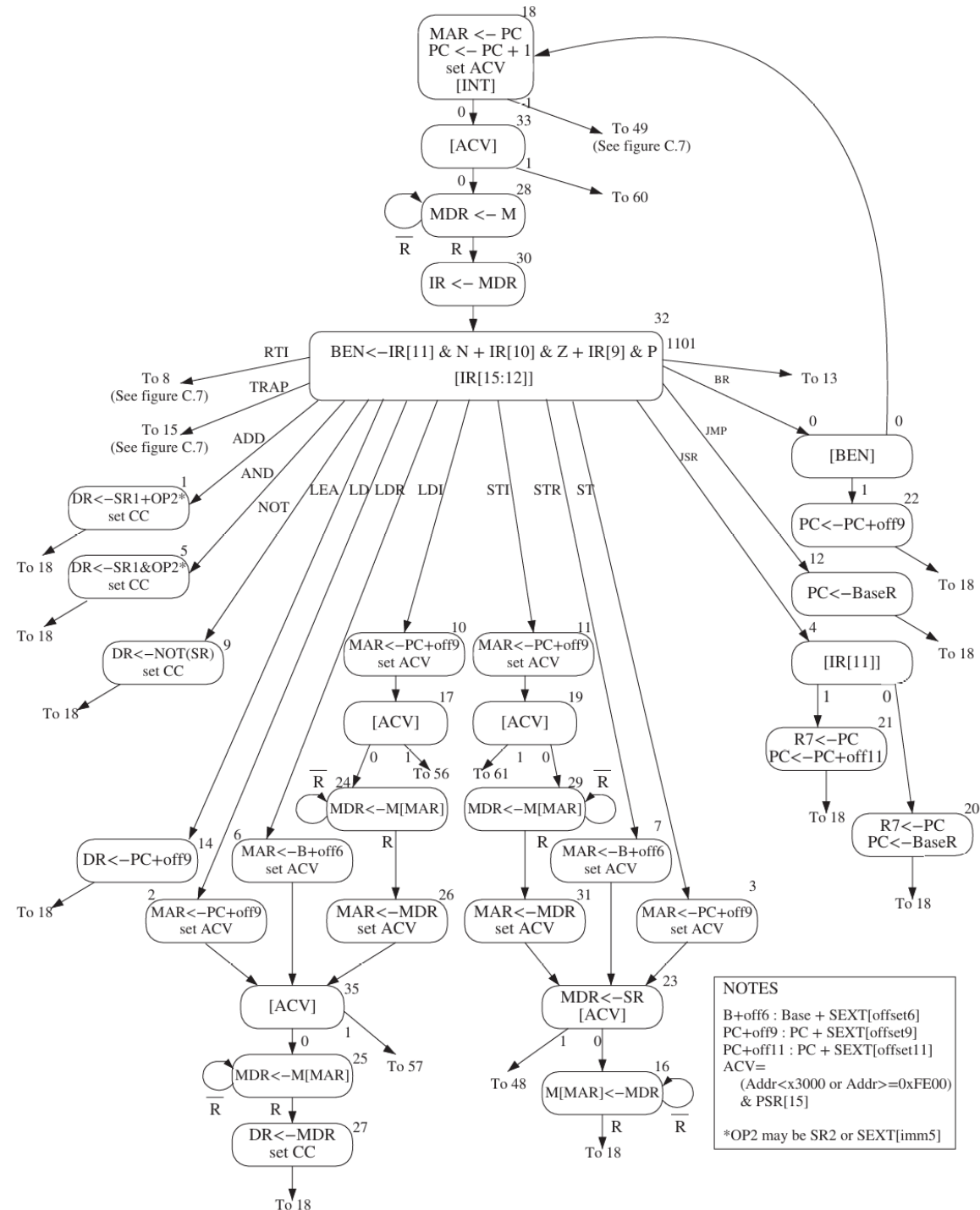
C/C++ 与 LC-3

- 指针和数组
 - 理解 C 语言中指针和数组对象的声明语法
 - 以数组或指针形式访问元素
 - 将 C 代码转换为 LC-3 实现

附录 C LC-3 微架构

• 完整的 LC-3 状态转换

- 一个圆圈表示一个状态，箭头表示状态的转换过程
- 题目问时钟周期 (cycle) ?
 - 访存的状态（指向自身的边），根据题目描述说多少就是多少
 - 其他状态，一个圆圈就是一个周期
- 题目给出单条指令时钟周期
 - 设一次访存需要的周期数为 x ，比如题目给出 ADD 指令运行的周期数是 10，ADD 指令一共是 6 个状态，其中一个状态是访存， $5 + x = 10$ ，那么一次访存的时钟周期数便是 5

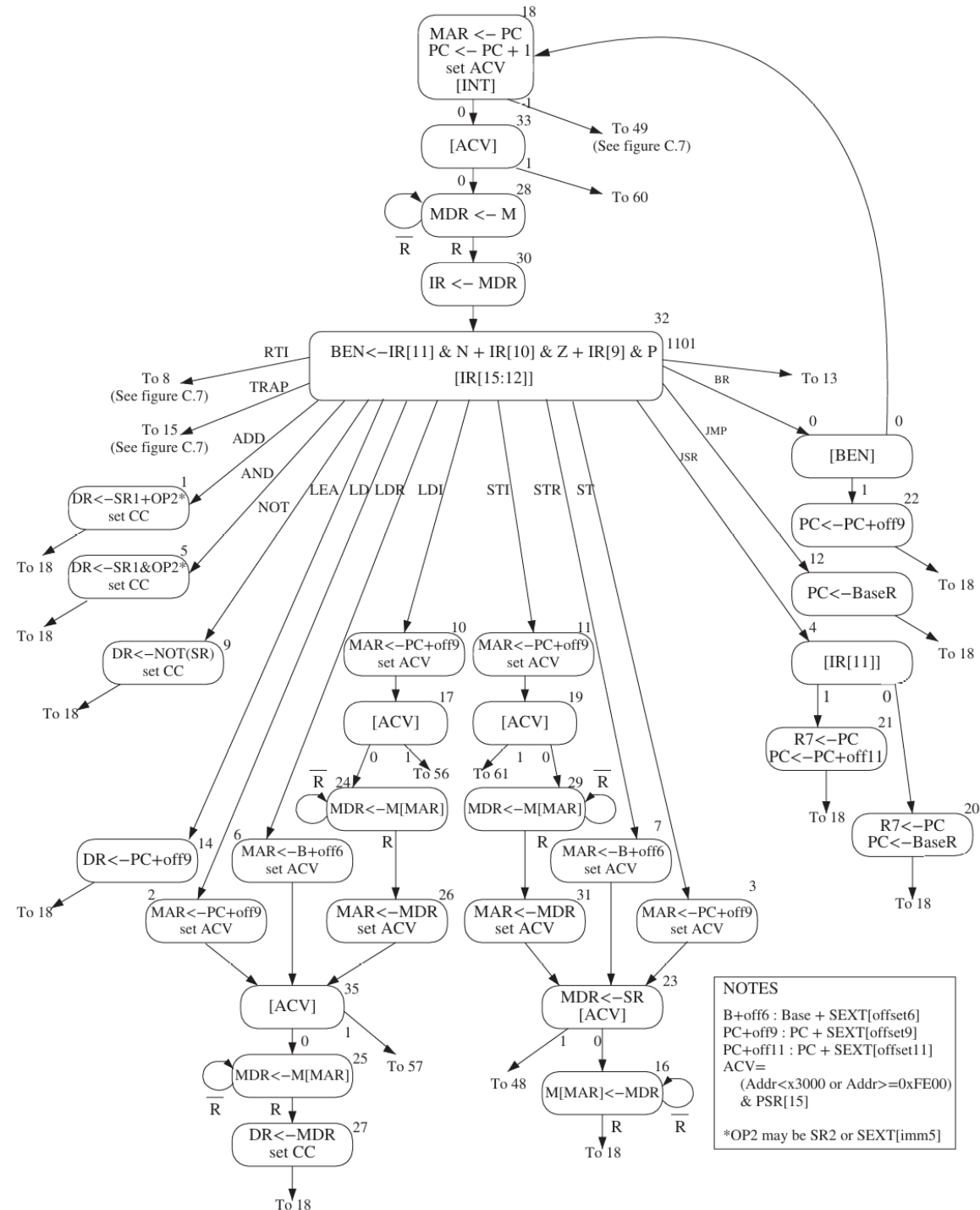


附录 C LC-3 微架构

• 完整的 LC-3 状态转换

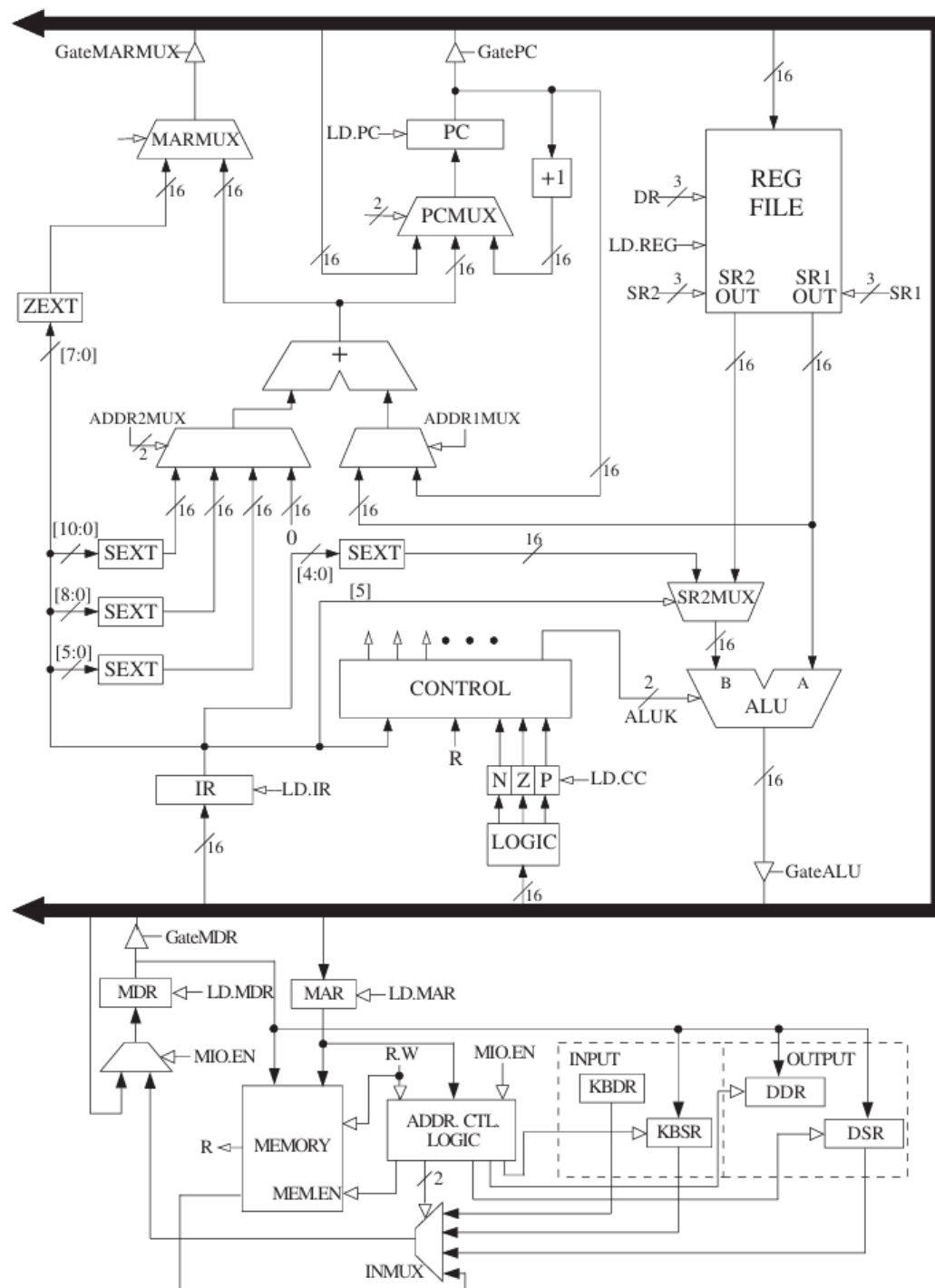
- 有了个这个状态机，能回答很多问题

- 哪些指令会改变 NZP 状态码？状态转换的过程中，包含 set CC 的状态会改变 NZP 状态码
- 哪些指令涉及到访存的过程？状态转换的过程中，涉及 M 的指令会访存



• LC-3 数据通路

- LD.XXX: 控制的 XXX 在当前状态是否需要装载值
- GateXXX: 控制 XXX 的输出是否要传输到总线上
- XXXMUX: 控制 XXX 的值来自于哪里
- ALUK: 控制ALU的行为, 可以是 ADD、AND、NOT、PASSA

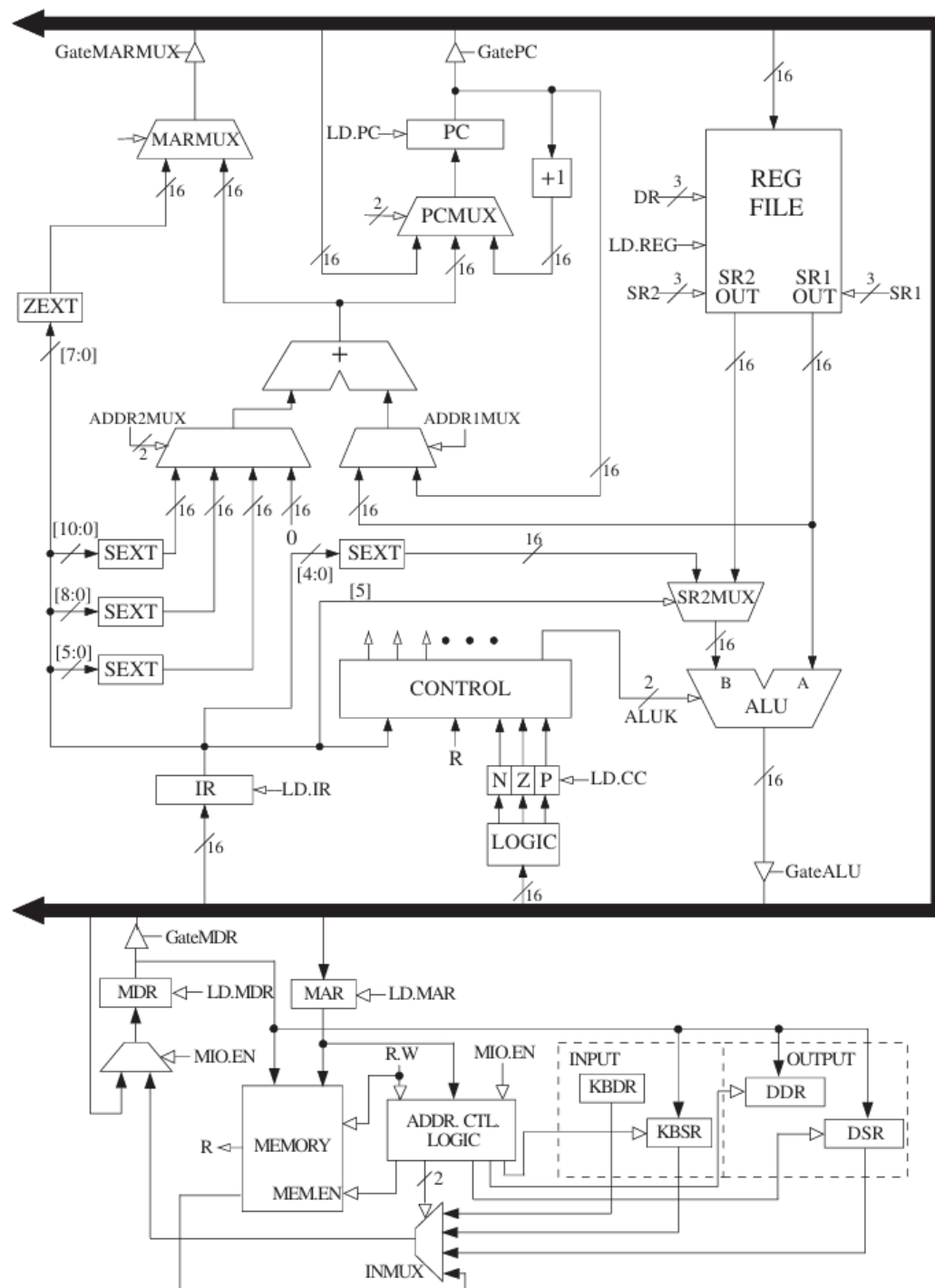


附录 C LC-3 微架构

• LC-3 数据通路

- 示例：当 ADD 指令执行到状态 1，且第二个操作数是立即数时：

- ALU 需要进行 ADD 运算，ALUK 的值是 ADD
- ALU 的结果需要输出到总线上，写到寄存器文件里
 - GateALU 为 YES
 - LD.REG 为 LOAD
- 计算的第二个操作数是立即数，所以 SR2MUX 的值应该是 IR[4:0]（经过符号扩展）



The following program copies a specified number (SIZE) of memory values from a source (SRC) to a destination (DST).

Do not count the HALT instruction when answering the questions.

(a) How many instructions are processed during the runtime period ?

(b) Assuming one memory access takes 10 clock cycles, how many clock cycles will the program take ?

```
1      .ORIG x300
2      LD  R2, DST
3      LD  R1, SRC
4      LD  R0, SIZE
5  LOOP BRz EXIT
6      ADD R0, R0, #-1
7      ADD R4, R1, R0
8      LDR R5, R4, #0
9      ADD R4, R2, R0
10     STR R5, R4, #0
11     ADD R0, R0, #0
12     BR  LOOP
13  EXIT HALT
14  SRC  .FILL x4000
15  DST  .FILL x5000
16  SIZE .FILL x10
17     .END
```

The following program copies a specified number (SIZE) of memory values from a source (SRC) to a destination (DST).

Do not count the HALT instruction when answering the questions.

(a) How many instructions are processed during the runtime period ?

注意迭代次数 x10 是十六进制，实际迭代了 16 次。

开始 3 条指令 + 16 次循环每次运行 8 条指令 + 最后一次跳出循环的判断 $3 + 8 \times 16 + 1 = 132$ 条。

(b) Assuming one memory access takes 10 clock cycles, how many clock cycles will the program take ?

```
1      .ORIG x300
2      LD  R2, DST
3      LD  R1, SRC
4      LD  R0, SIZE
5  LOOP BRz EXIT
6      ADD R0, R0, #-1
7      ADD R4, R1, R0
8      LDR R5, R4, #0
9      ADD R4, R2, R0
10     STR R5, R4, #0
11     ADD R0, R0, #0
12     BR LOOP
13  EXIT HALT
14  SRC  .FILL x4000
15  DST  .FILL x5000
16  SIZE .FILL x10
17     .END
```

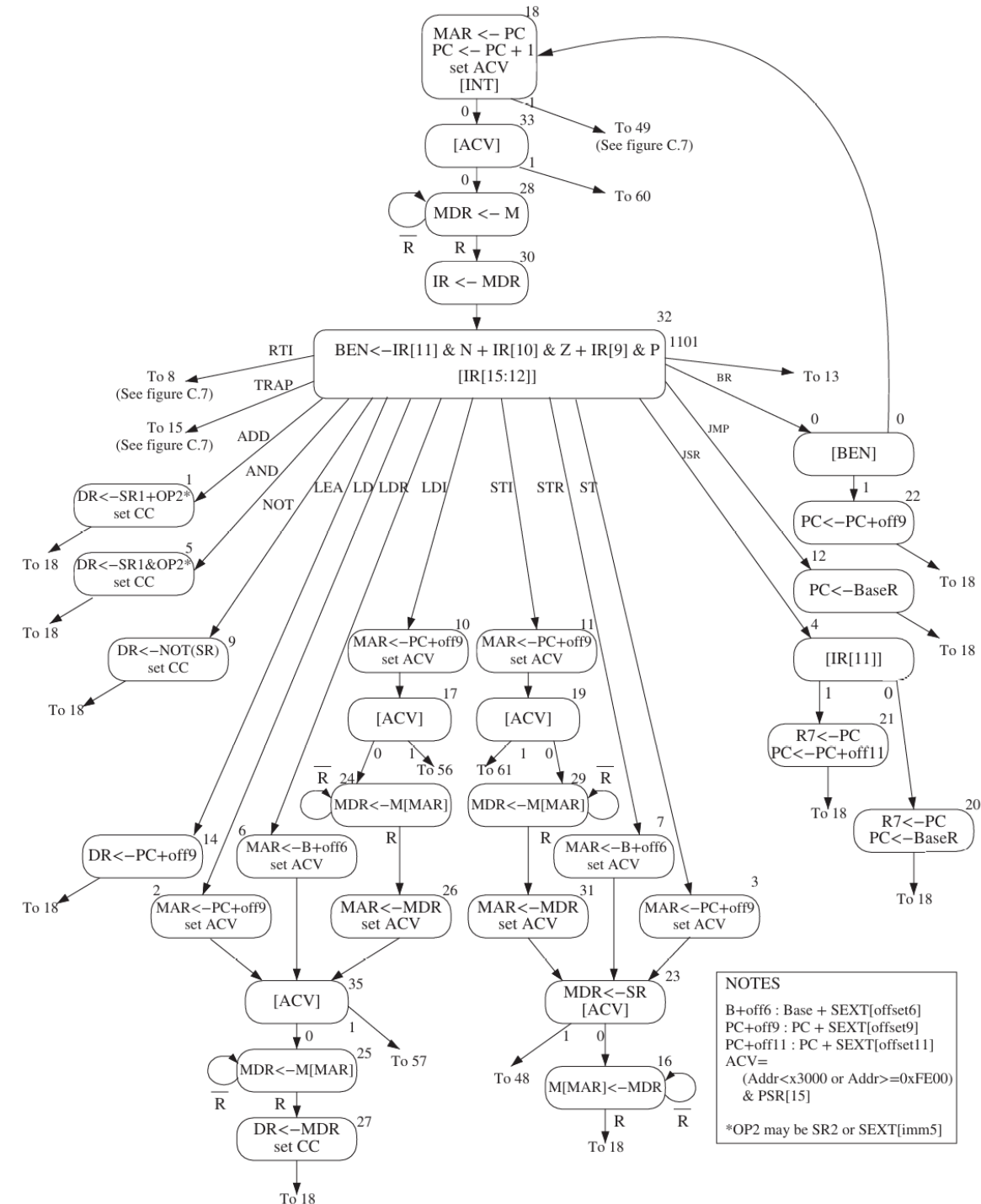
The following program copies a specified number (SIZE) of memory values from a source (SRC) to a destination (DST).

Do not count the HALT instruction when answering the questions.

(a) How many instruction are processed during the runtime period ?

(b) Assuming one memory access takes 10 clock cycles, how many clock cycles will the program take ?

指令	周期



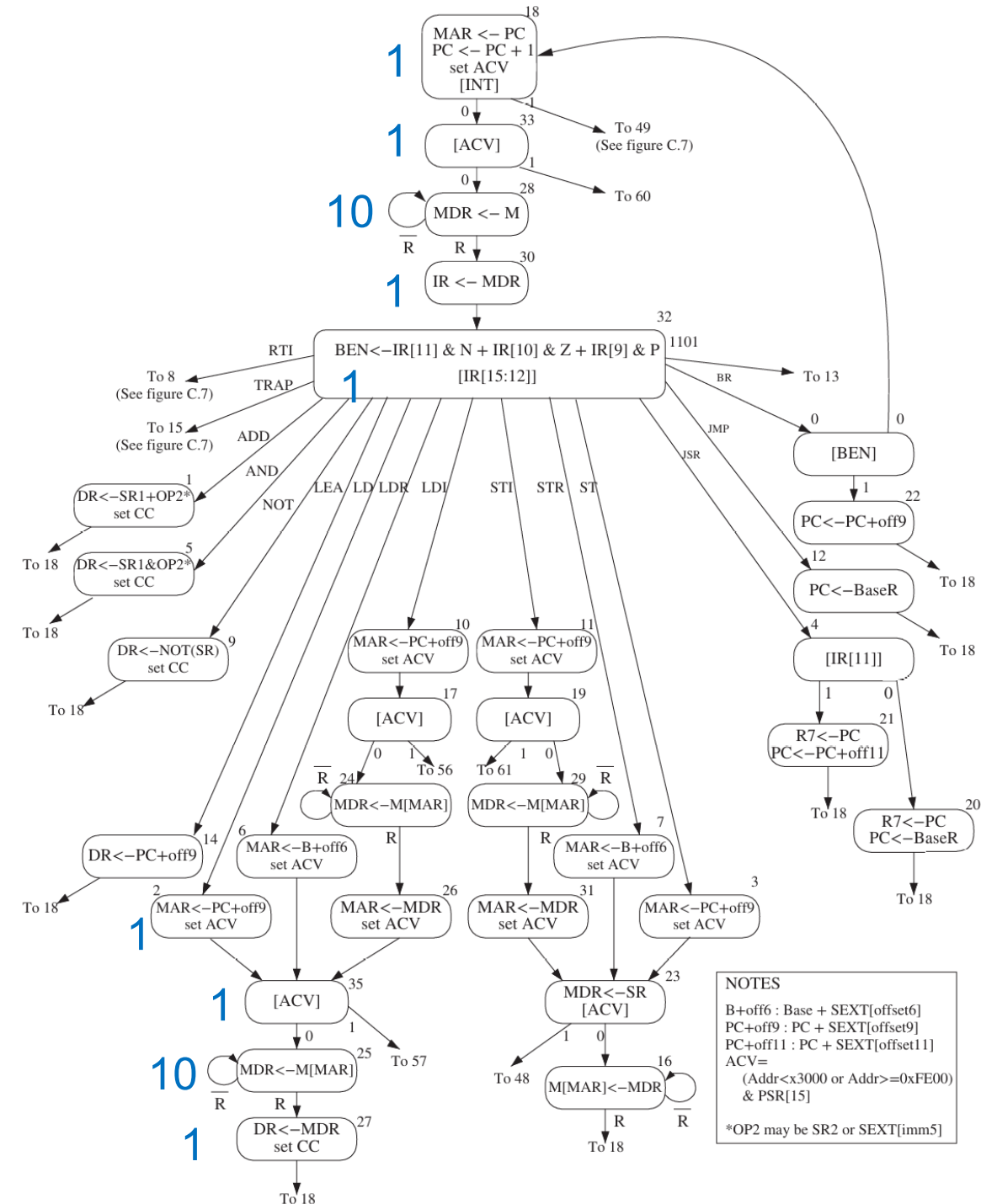
The following program copies a specified number (SIZE) of memory values from a source (SRC) to a destination (DST).

Do not count the HALT instruction when answering the questions.

(a) How many instruction are processed during the runtime period ?

(b) Assuming one memory access takes 10 clock cycles, how many clock cycles will the program take ?

指令	周期
LD	27
LDR	27



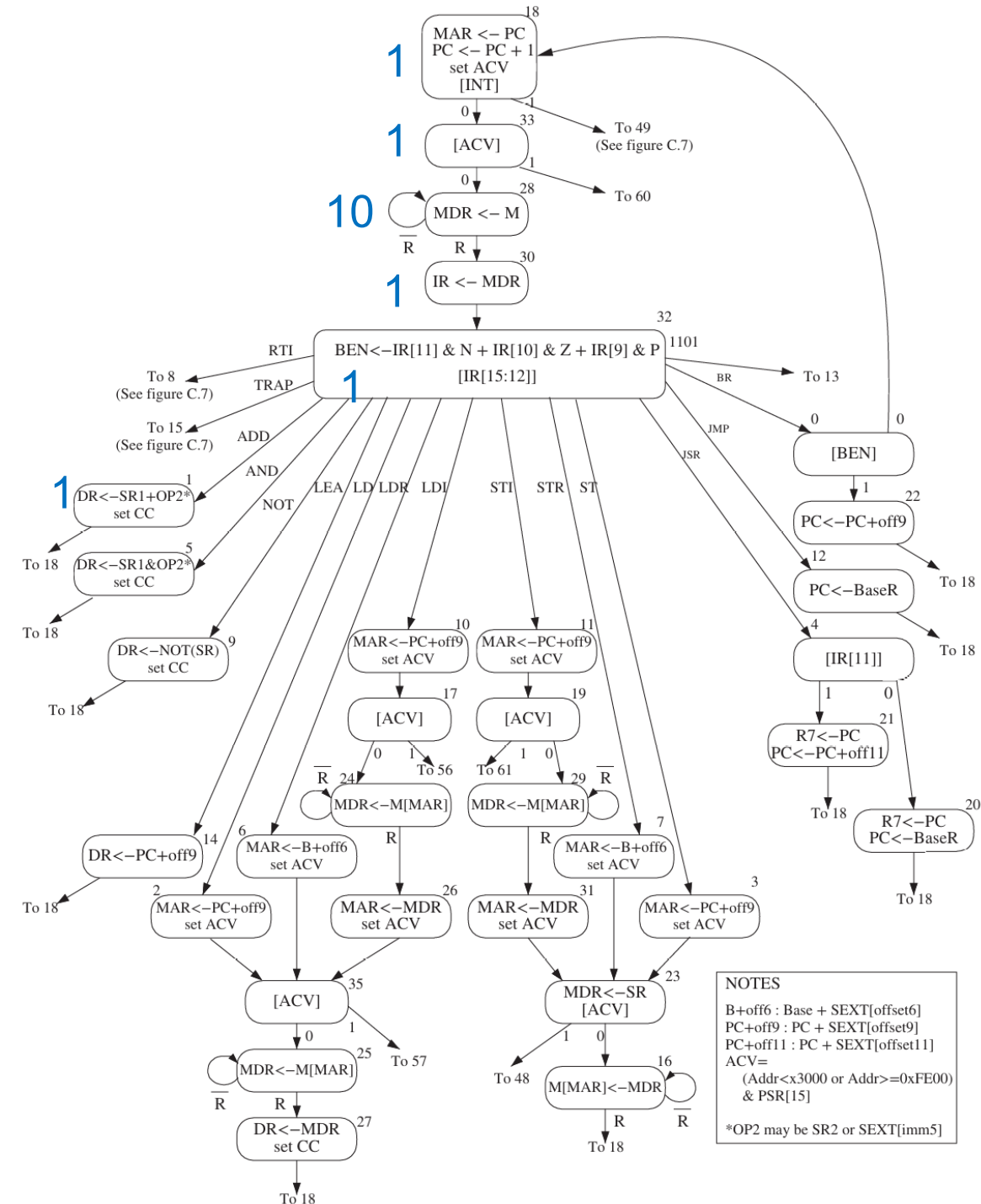
The following program copies a specified number (SIZE) of memory values from a source (SRC) to a destination (DST).

Do not count the HALT instruction when answering the questions.

(a) How many instruction are processed during the runtime period ?

(b) Assuming one memory access takes 10 clock cycles, how many clock cycles will the program take ?

指令	周期
LD	27
LDR	27
ADD	15



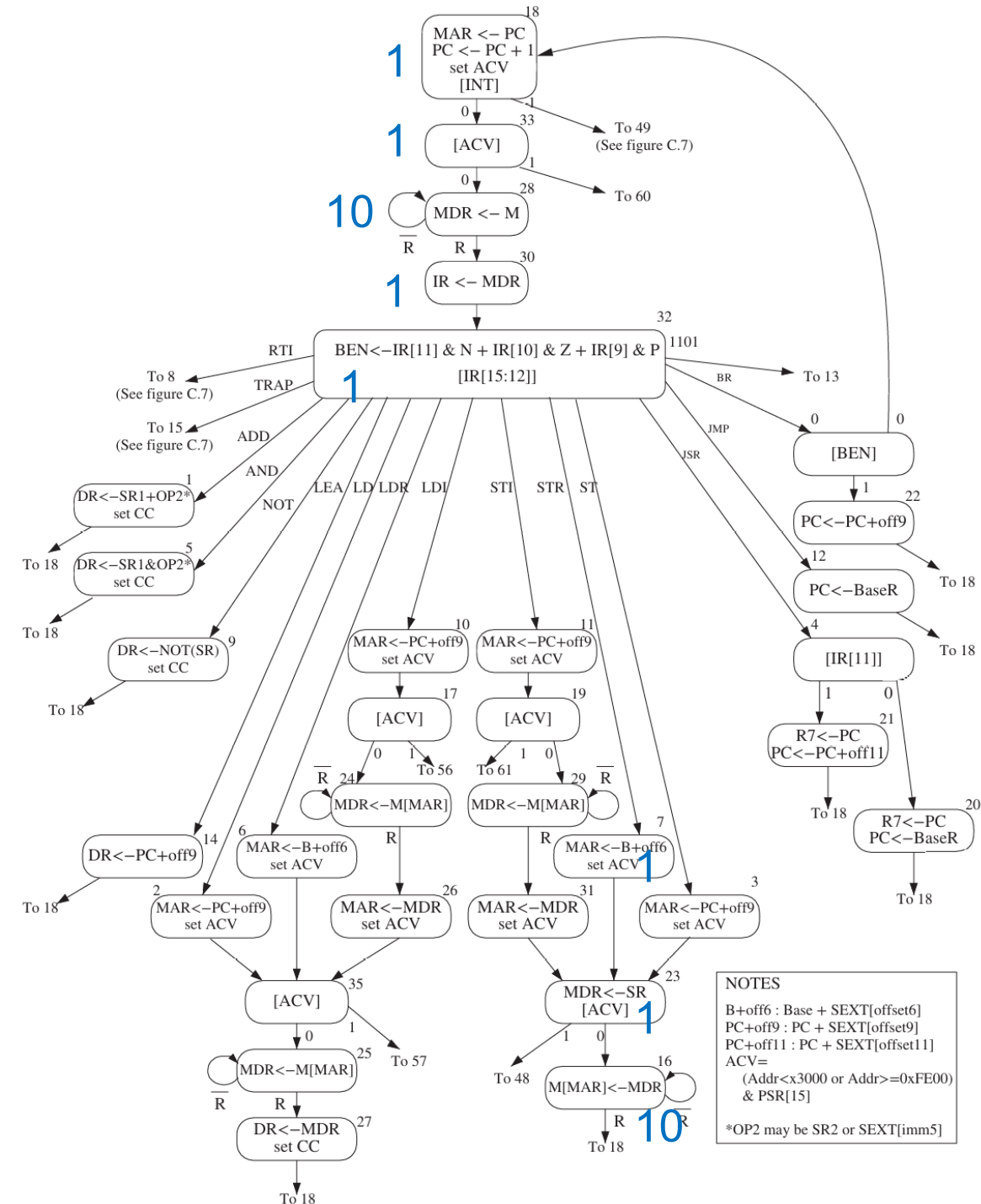
The following program copies a specified number (SIZE) of memory values from a source (SRC) to a destination (DST).

Do not count the HALT instruction when answering the questions.

(a) How many instruction are processed during the runtime period ?

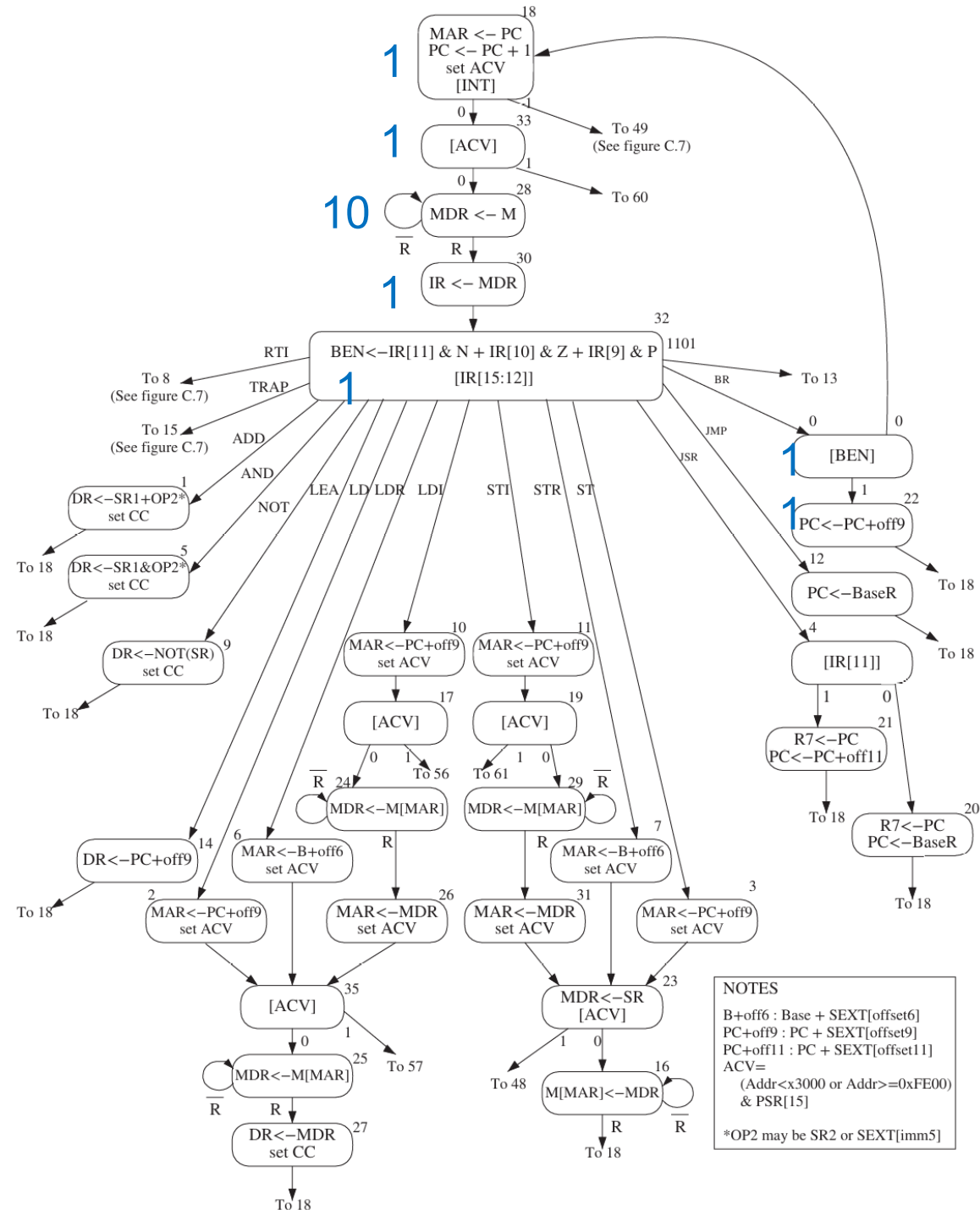
(b) Assuming one memory access takes 10 clock cycles, how many clock cycles will the program take ?

指令	周期
LD	27
LDR	27
ADD	15
STR	26



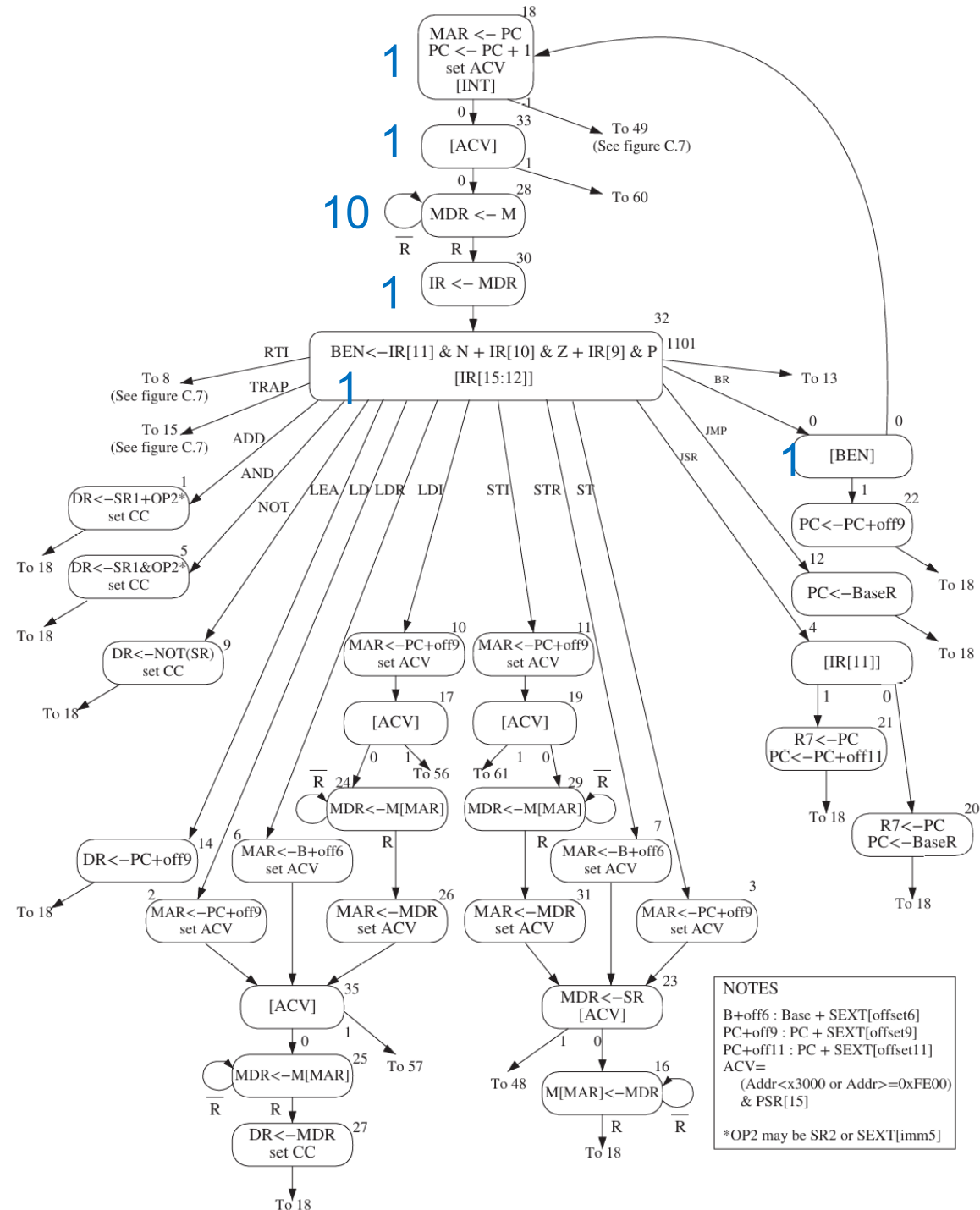
(b) Assuming one memory access takes 10 clock cycles, how many clock cycles will the program take ?

指令	周期
LD	27
LDR	27
ADD	15
STR	26
BR（跳转）	16



(b) Assuming one memory access takes 10 clock cycles, how many clock cycles will the program take ?

指令	周期
LD	27
LDR	27
ADD	15
STR	26
BR（跳转）	16
BR（无跳转）	15



The following program copies a specified number (SIZE) of memory values from a source (SRC) to a destination (DST).

Do not count the HALT instruction when answering the questions.

(a) How many instruction are processed during the runtime period ?

(b) Assuming one memory access takes 10 clock cycles, how many clock cycles will the program take ?

循环：1 个不跳转的 BR + 1 个跳转的 BR + 4 个 ADD + 1 个 LDR + 1 个 STR。

共 $15 + 16 + 4 \times 15 + 26 + 26 = 144$ 周期。

最后总的周期是：3 个 LD + 16 次循环 + 1 个跳转的 BR： $3 \times 27 + 144 \times 16 + 16 = 2401$

指令	周期
LD	27
LDR	27
ADD	15
STR	26
BR (跳转)	16
BR (无跳转)	15

```
1      .ORIG x300
2      LD  R2, DST
3      LD  R1, SRC
4      LD  R0, SIZE
5      LOOP BRz EXIT
6      ADD R0, R0, #-1
7      ADD R4, R1, R0
8      LDR R5, R4, #0
9      ADD R4, R2, R0
10     STR R5, R4, #0
11     ADD R0, R0, #0
12     BR LOOP
13     EXIT HALT
14     SRC  .FILL x4000
15     DST  .FILL x5000
16     SIZE .FILL x10
17     .END
```

We have a program with some missing instructions, and we have a table consisting of some information and some missing information associated with five specific clock cycles of the program's execution. Your job is to complete both!

(1) Insert the missing instructions in the program and the missing information in the table. Cycle numbering starts at 1. That is, cycle 1 is the first clock cycle of the processing of LD R0, A. Note that we are asking for the value of the registers DURING each clock cycle. We have not said anything about the number of clock cycles a memory access takes, but you do have enough information to figure that out for yourself.

```

1      .ORIG x3000
2      LD R0, A
3      LD R1, B
4      NOT R1, R1
5      ADD R1, R1, #1
6      AND R2, R2, #0
7  AGAIN ----- (a)
8      ----- (b)
9      BRnzp AGAIN
10     DONE ST R2, C
11     HALT
12     A    .FILL #5
13     B    .FILL ----- (c)
14     C    .BLKW #1
15     .END

```

Cycle Number	State Number	Information			
(d)	(e)	LD.REG: 1	DRMUX: (f)	GateMDR: (g)	
		LD.CC: (h)	GateALU: (i)	GatePC: (j)	
16	30	LD.MDR: (k)	MDR: (m)	IR: (n)	
		LD.IR: (l)			
50	(o)	LD.REG: 1	MDR: (p)	DRMUX: (q)	
		BUS: x0001	x_4A_	GateMDR: (r)	
57	1	PC: (s)	IR: (t)	GateALU: (u)	
		BUS: x0003	x_040	GatePC: (v)	
(w)	22	ADDR1MUX: (x)	ADDR2MUX: (y)		
		LD.PC: 1	PC: x3008	PCMUX: ADDER	

(1) Insert the missing instructions in the program and the missir
That is, cycle 1 is the first clock cycle of the processing of LD R
DURING each clock cycle. We have not said anything about the
do have enough information to figure that out for yourself.

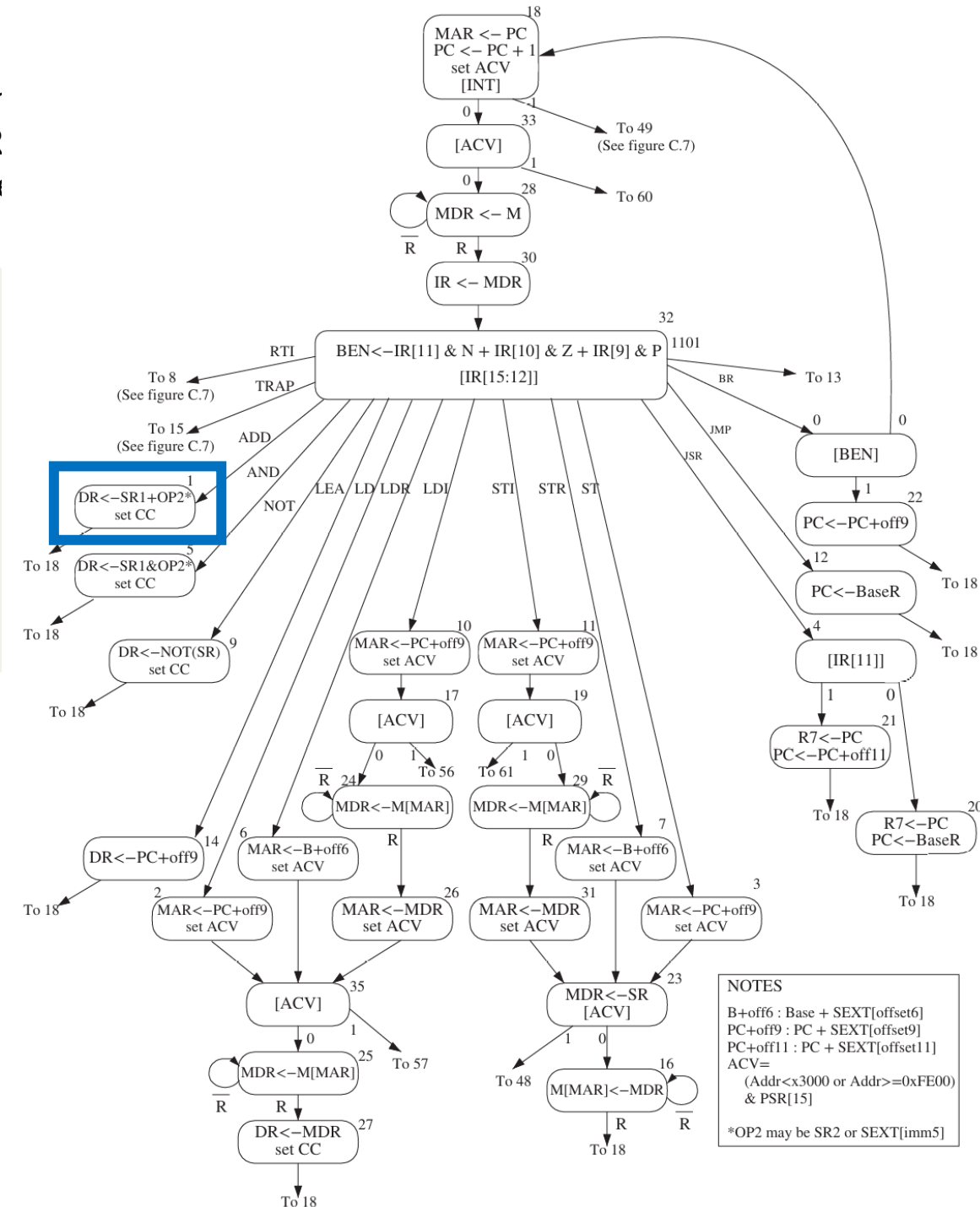
```

1 .ORIG x3000
2 LD R0, A
3 LD R1, B
4 NOT R1, R1
5 ADD R1, R1, #1
6 AND R2, R2, #0
7 AGAIN ----- (a)
8 ----- (b)
9 BRnzp AGAIN
10 DONE ST R2, C
11 HALT
12 A .FILL #5
13 B .FILL ----- (c)
14 C .BLKW #1
15 .END

```

确定这是 ADD 指令，即 x1040
(ADD R0, R1, R0)

Cycle Number	State Number	Information			
(d)	(e)	LD.REG: 1	DRMUX: (f)	GateMDR: (g)	
		LD.CC: (h)	GateALU: (i)	GatePC: (j)	
16	30	LD.MDR: (k)	MDR: (m)	IR: (n)	
		LD.IR: (l)			
50	(o)	LD.REG: 1	MDR: (p) x_4A_	DRMUX: (q)	
		BUS: x0001		GateMDR: (r)	
57	1	PC: (s)	IR: (t) x_040	GateALU: (u)	
		BUS: x0003		GatePC: (v)	
(w)	22	ADDR1MUX: (x)	ADDR2MUX: (y)		
		LD.PC: 1	PC: x3008	PCMUX: ADDER	



(1) Insert the missing instructions in the program and the missir
That is, cycle 1 is the first clock cycle of the processing of LD R
DURING each clock cycle. We have not said anything about the
do have enough information to figure that out for yourself.

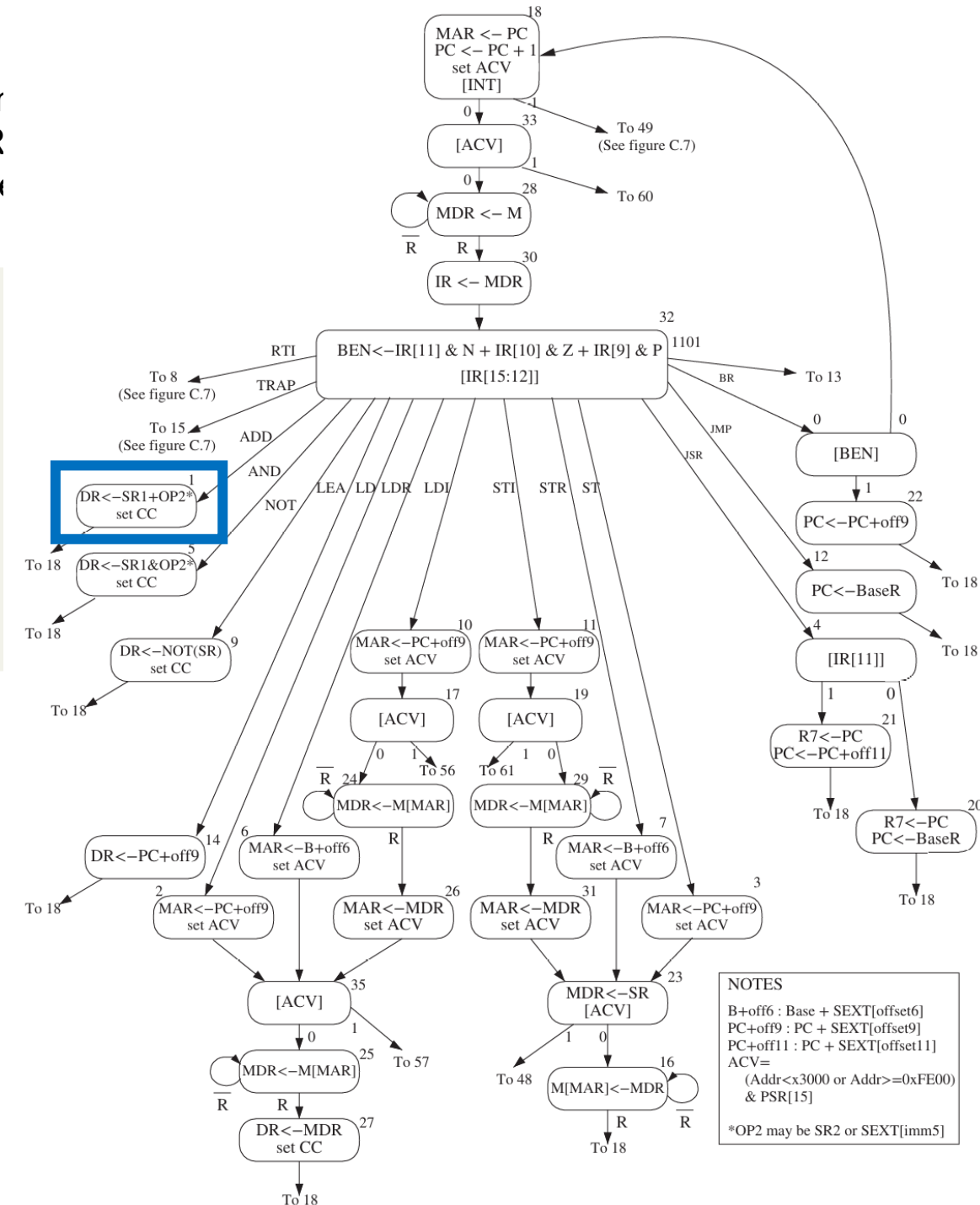
```

.Orig x3000
2 LD R0, A
3 LD R1, B
4 NOT R1, R1
5 ADD R1, R1, #1
6 AND R2, R2, #0
7 AGAIN ----- (a)
8 ----- (b)
9 BRnzp AGAIN
10 DONE ST R2, C
11 HALT
12 A .FILL #5
13 B .FILL ----- (c)
14 C .BLKW #1
15 .END

```

Cycle Number	State Number	Information			
(d)	(e)	LD.REG: 1	DRMUX: (f)	GateMDR: (g)	
		LD.CC: (h)	GateALU: (i)	GatePC: (j)	
16	30	LD.MDR: (k)	MDR: (m)	IR: (n)	
		LD.IR: (l)			
50	(o)	LD.REG: 1	MDR: (p)	DRMUX: (q)	
		BUS: x0001	x_4A_	GateMDR: (r)	
57	1	PC: (s)	IR: (t)	GateALU: 1 (u)	
		BUS: x0003	x1040	GatePC: 0 (v)	
(w)	22	ADDR1MUX: (x)	ADDR2MUX: (y)		
		LD.PC: 1	PC: x3008	PCMUX: ADDER	

ADD R0, R1, R0



(1) Insert the missing instructions in the program and the missir
That is, cycle 1 is the first clock cycle of the processing of LD R
DURING each clock cycle. We have not said anything about the
do have enough information to figure that out for yourself.

```

.Orig x3000
LD R0, A
LD R1, B
NOT R1, R1
ADD R1, R1, #1
AND R2, R2, #0
AGAIN ----- (a)
----- (b)
BRnzp AGAIN
DONE ST R2, C
HALT
A .FILL #5
B .FILL ----- (c)
C .BLKW #1
.END

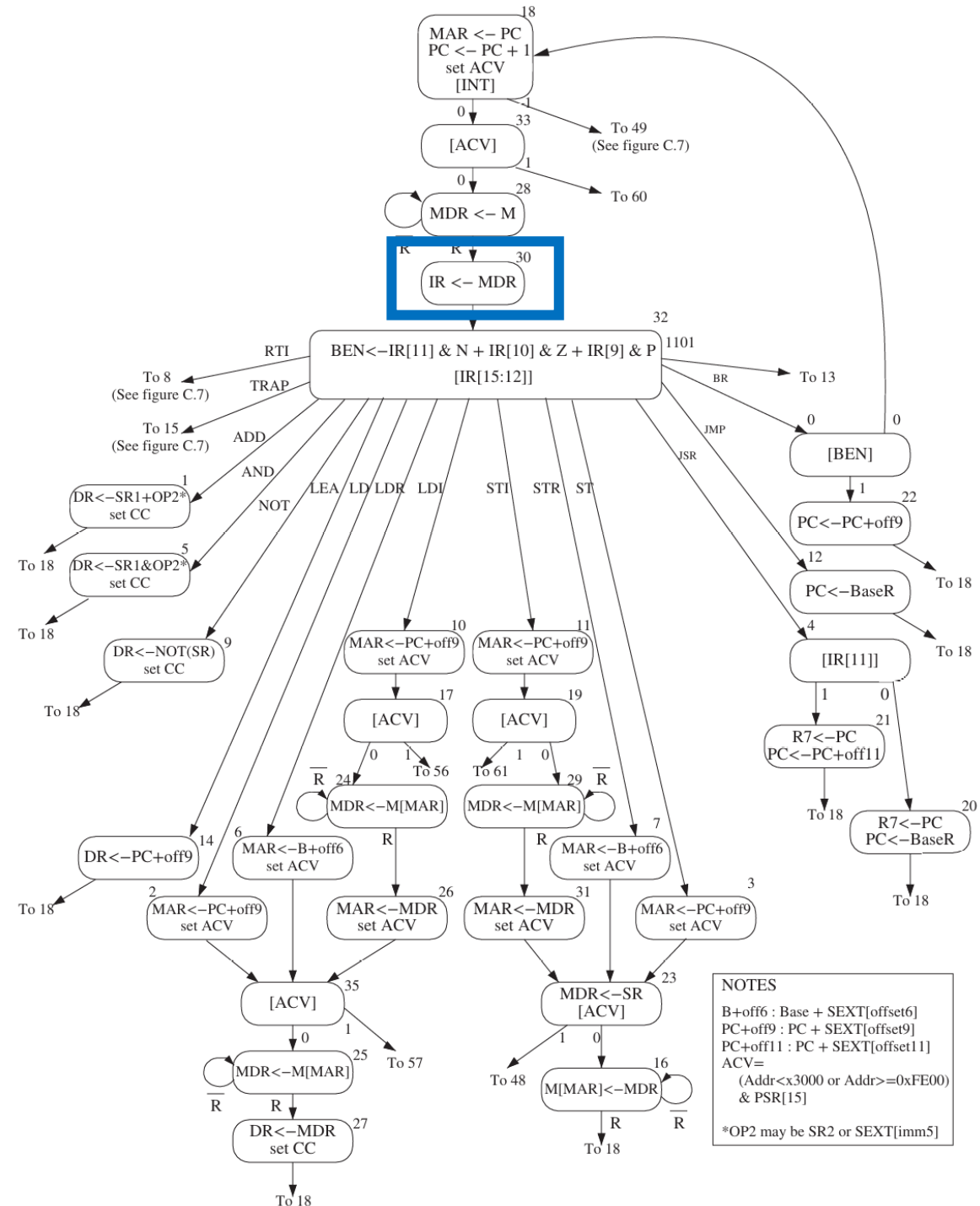
```

如果访存需要 x 周期，则 LD 指令
需要 7 + 2x 周期。
解得 x = 2，即访存需要 2 周期。

只能是第二条指令
LD R1, B

ADD R0, R1, R0

Cycle Number	State Number	Information			
(d)	(e)	LD.REG: 1	DRMUX: (f)	GateMDR: (g)	
		LD.CC: (h)	GateALU: (i)	GatePC: (j)	
16	30	LD.MDR: (k)	MDR: (m)	IR: (n)	
		LD.IR: (l)			
50	(o)	LD.REG: 1	MDR: (p) x_4A_	DRMUX: (q)	
		BUS: x0001		GateMDR: (r)	
57	1	PC: (s)	IR: (t) x1040	GateALU: 1 (u)	
		BUS: x0003		GatePC: 0 (v)	
(w)	22	ADDR1MUX: (x)	ADDR2MUX: (y)		
		LD.PC: 1	PC: x3008	PCMUX: ADDER	



(1) Insert the missing instructions in the program and the missir
That is, cycle 1 is the first clock cycle of the processing of LD R
DURING each clock cycle. We have not said anything about the
do have enough information to figure that out for yourself.

```

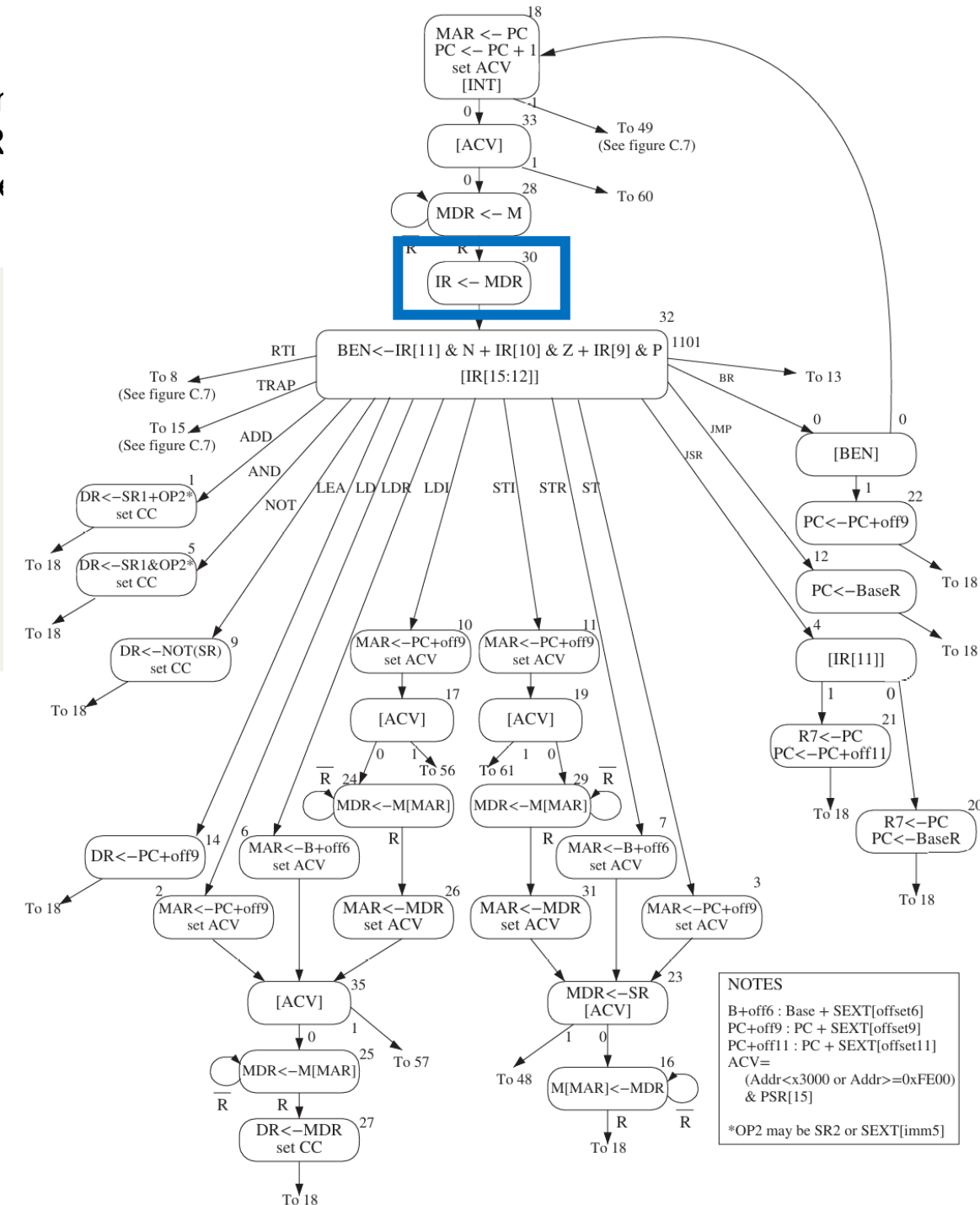
1 .ORIG x3000
2 LD R0, A
3 LD R1, B
4 NOT R1, R1
5 ADD R1, R1, #1
6 AND R2, R2, #0
7 AGAIN ----- (a)
8 ----- (b)
9 BRnzp AGAIN
10 DONE ST R2, C
11 HALT
12 A .FILL #5
13 B .FILL ----- (c)
14 C .BLKW #1
15 .END

```

LD R1, B

ADD R0, R1, R0

Cycle Number	State Number	Information			
(d)	(e)	LD.REG: 1	DRMUX: (f)	GateMDR: (g)	
		LD.CC: (h)	GateALU: (i)	GatePC: (j)	
16	30	LD.MDR: 0 (k)	MDR: (m)	IR: (n)	
		LD.IR: 1 (l)	x2209	x2009	
50	(o)	LD.REG: 1	MDR: (p)	DRMUX: (q)	
		BUS: x0001	x_4A_	GateMDR: (r)	
57	1	PC: (s)	IR: (t)	GateALU: 1 (u)	
		BUS: x0003	x1040	GatePC: 0 (v)	
(w)	22	ADDR1MUX: (x)	ADDR2MUX: (y)		
		LD.PC: 1	PC: x3008	PCMUX: ADDER	



(1) Insert the missing instructions in the program and the missir
That is, cycle 1 is the first clock cycle of the processing of LD R
DURING each clock cycle. We have not said anything about the
do have enough information to figure that out for yourself.

```

.Orig x3000
LD R0, A
LD R1, B
NOT R1, R1
ADD R1, R1, #1
AND R2, R2, #0
AGAIN ----- (a)
----- (b)
BRnzp AGAIN
DONE ST R2, C
HALT
A .FILL #5
B .FILL ----- (c)
C .BLKW #1
.END

```

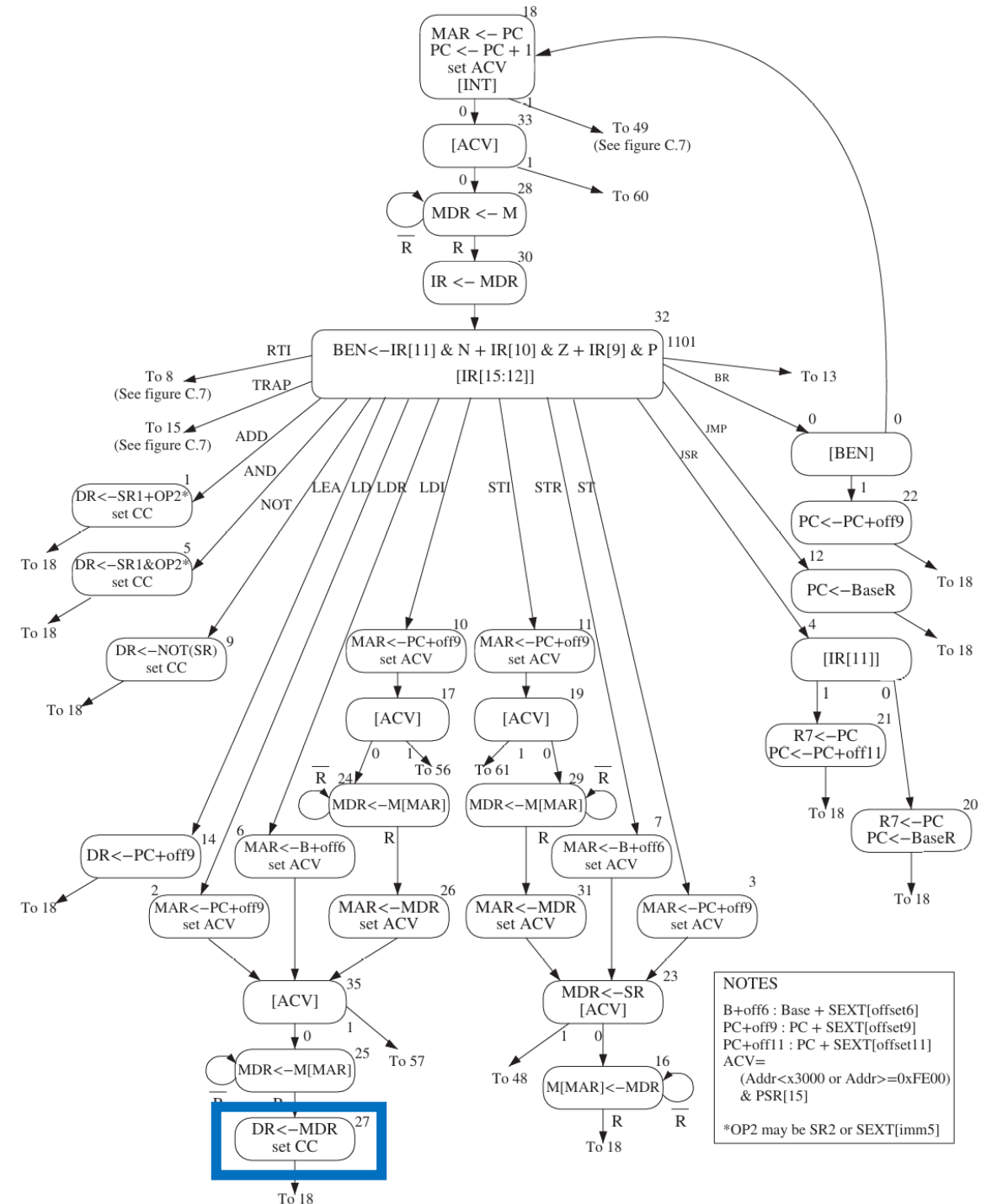
LD 指令中只有状态 27 需要写入寄存器。

LD R1, A

LD R1, B

ADD R0, R1, R0

Cycle Number	State Number	Information			
(d)	(e)	LD.REG: 1	DRMUX: (f)	GateMDR: (g)	
		LD.CC: (h)	GateALU: (i)	GatePC: (j)	
16	30	LD.MDR: 0 (k)	MDR: (m)	IR: (n)	
		LD.IR: 1 (l)	x2209	x2009	
50	(o)	LD.REG: 1	MDR: (p)	DRMUX: (q)	
		BUS: x0001	x_4A_	GateMDR: (r)	
57	1	PC: (s)	IR: (t)	GateALU: 1 (u)	
		BUS: x0003	x1040	GatePC: 0 (v)	
(w)	22	ADDR1MUX: (x)	ADDR2MUX: (y)		
		LD.PC: 1	PC: x3008	PCMUX: ADDER	



(1) Insert the missing instructions in the program and the missir
That is, cycle 1 is the first clock cycle of the processing of LD R
DURING each clock cycle. We have not said anything about the
do have enough information to figure that out for yourself.

```

.Orig x3000
2 LD R0, A
3 LD R1, B
4 NOT R1, R1
5 ADD R1, R1, #1
6 AND R2, R2, #0
7 AGAIN ----- (a)
8 ----- (b)
9 BRnzp AGAIN
10 DONE ST R2, C
11 HALT
12 A .FILL #5
13 B .FILL ----- (c)
14 C .BLKW #1
15 .END

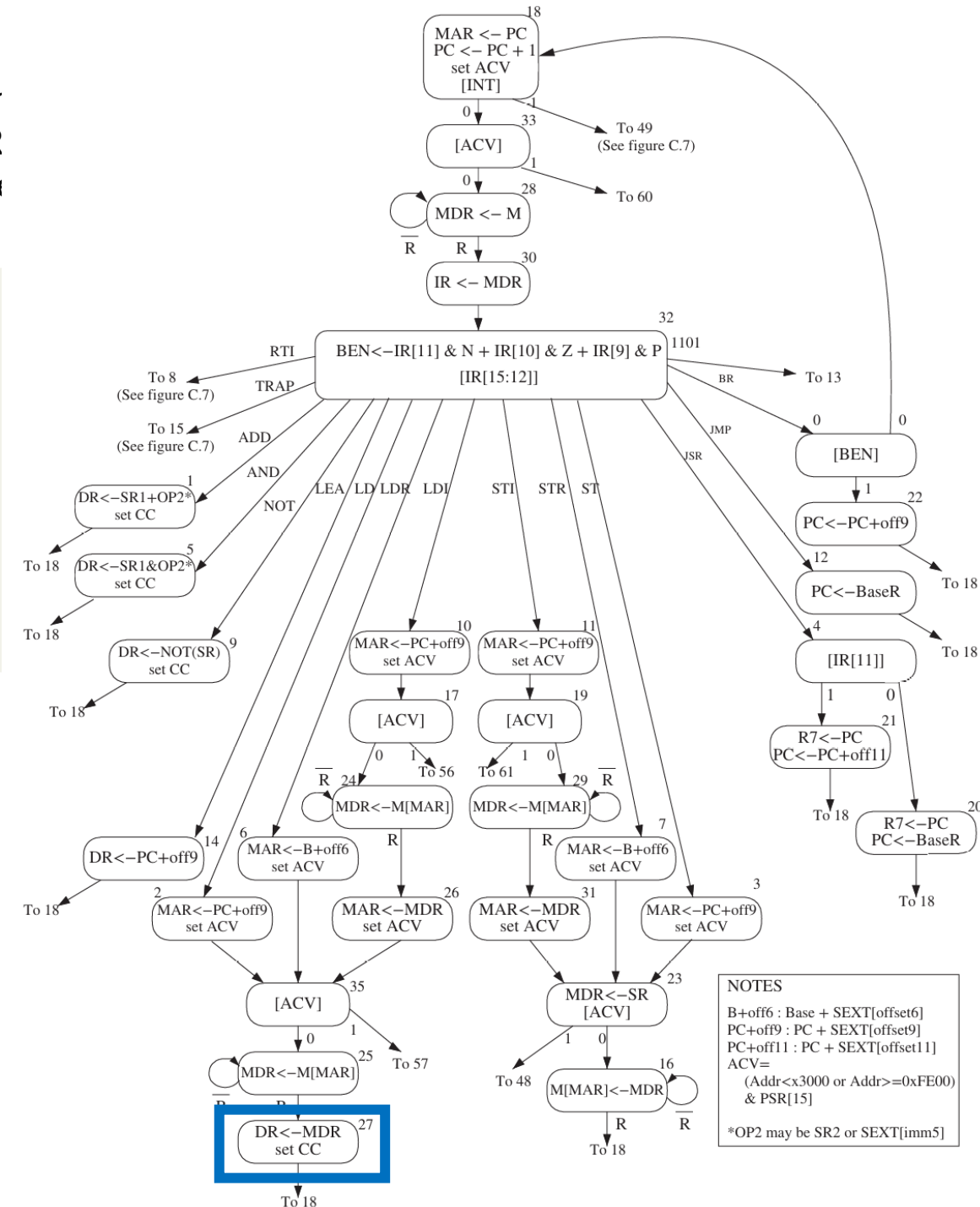
```

LD R1, A

LD R1, B

ADD R0, R1, R0

Cycle Number	State Number	Information			
(d) 11	(e) 27	LD.REG: 1	DRMUX: 0 (f)	GateMDR: 1 (g)	
		LD.CC: 1 (h)	GateALU: 0 (i)	GatePC: 0 (j)	
16	30	LD.MDR: 0 (k)	MDR: (m) x2209	IR: (n) x2009	
		LD.IR: 1 (l)			
50	(o)	LD.REG: 1	MDR: (p) x_4A_	DRMUX: (q)	
		BUS: x0001		GateMDR: (r)	
57	1	PC: (s)	IR: (t) x1040	GateALU: 1 (u)	
		BUS: x0003		GatePC: 0 (v)	
(w)	22	ADDR1MUX: (x)	ADDR2MUX: (y)		
		LD.PC: 1	PC: x3008	PCMUX: ADDER	



(1) Insert the missing instructions in the program and the missing information in the table. Cycle numbering starts at 1. That is, cycle 1 is the first clock cycle of the processing of LD R0, A. Note that we are asking for the value of the registers DURING each clock cycle. We have not said anything about the number of clock cycles a memory access takes, but you do have enough information to figure that out for yourself.

```

11 LD R0, A
LD R1, B
NOT R1, R1
7 ADD R1, R1, #1
AND R2, R2, #0
AGAIN ----- (a)
----- (b)
BRnzp AGAIN
DONE ST R2, C
HALT
A .FILL #5
B .FILL ----- (c)
C .BLKW #1
.END
```

确定访存周期数后，可以推断每条指令的执行周期。

LD R1, A
LD R1, B
ADD R0, R1, R0

Cycle Number	State Number	Information		
(d) 11	(e) 27	LD.REG: 1	DRMUX: 0 (f)	GateMDR: 1 (g)
		LD.CC: 1 (h)	GateALU: 0 (i)	GatePC: 0 (j)
16	30	LD.MDR: 0 (k)	MDR: (m) x2209	IR: (n) x2009
		LD.IR: 1 (l)		
50	(o)	LD.REG: 1	MDR: (p) x_4A_	DRMUX: (q)
		BUS: x0001	GateMDR: (r)	
57	1	PC: (s)	IR: (t) x1040	GateALU: 1 (u)
		BUS: x0003	GatePC: 0 (v)	
(w)	22	ADDR1MUX: (x)	ADDR2MUX: (y)	
		LD.PC: 1	PC: x3008	PCMUX: ADDER

(1) Insert the missing instructions in the program and the missing information in the table. Cycle numbering starts at 1. That is, cycle 1 is the first clock cycle of the processing of LD R0, A. Note that we are asking for the value of the registers DURING each clock cycle. We have not said anything about the number of clock cycles a memory access takes, but you do have enough information to figure that out for yourself.

```

15
14 C      .BLKW #1
13 B      .FILL ----- (c)
12 A      .FILL #5
11 DONE   BRnzp AGAIN
10        ST R2, C
9         HALT
8         ----- (b)
7 AGAIN   ----- (a)
6         AND R2, R2, #0
5         ADD R1, R1, #1
4         NOT R1, R1
3         LD R1, B
2         LD R0, A
1         .ORIG x3000

```

这是一条只需 7 周期的指令，推测是非访存指令（ $5 + 2 = 7$ ）。
该指令将 R2 和立即数运算后，结果写回 R2。
总线上的值是 x0001，推测是 ADD R2, R2, #1

LD R1, A

LD R1, B

ADD R0, R1, R0

Cycle Number	State Number	Information		
(d) 11	(e) 27	LD.REG: 1	DRMUX: 0 (f)	GateMDR: 1 (g)
		LD.CC: 1 (h)	GateALU: 0 (i)	GatePC: 0 (j)
16	30	LD.MDR: 0 (k)	MDR: (m) x2209	IR: (n) x2009
		LD.IR: 1 (l)		
50	(o)	LD.REG: 1	MDR: (p) x_4A_	DRMUX: (q)
		BUS: x0001		GateMDR: (r)
57	1	PC: (s)	IR: (t) 0100 1010 x1040	GateALU: 1 (u)
		BUS: x0003		GatePC: 0 (v)
(w)	22	ADDR1MUX: (x)	ADDR2MUX: (y)	
		LD.PC: 1	PC: x3008	PCMUX: ADDER

(1) Insert the missing instructions in the program and the missir
That is, cycle 1 is the first clock cycle of the processing of LD R
DURING each clock cycle. We have not said anything about the
do have enough information to figure that out for yourself.

```

.Orig x3000
11 LD R0, A
LD R1, B
NOT R1, R1
7 ADD R1, R1, #1
AND R2, R2, #0
AGAIN ----- (a)
----- (b)
BRnzp AGAIN
DONE ST R2, C
HALT
A .FILL #5
B .FILL ----- (c)
C .BLKW #1
.END

```

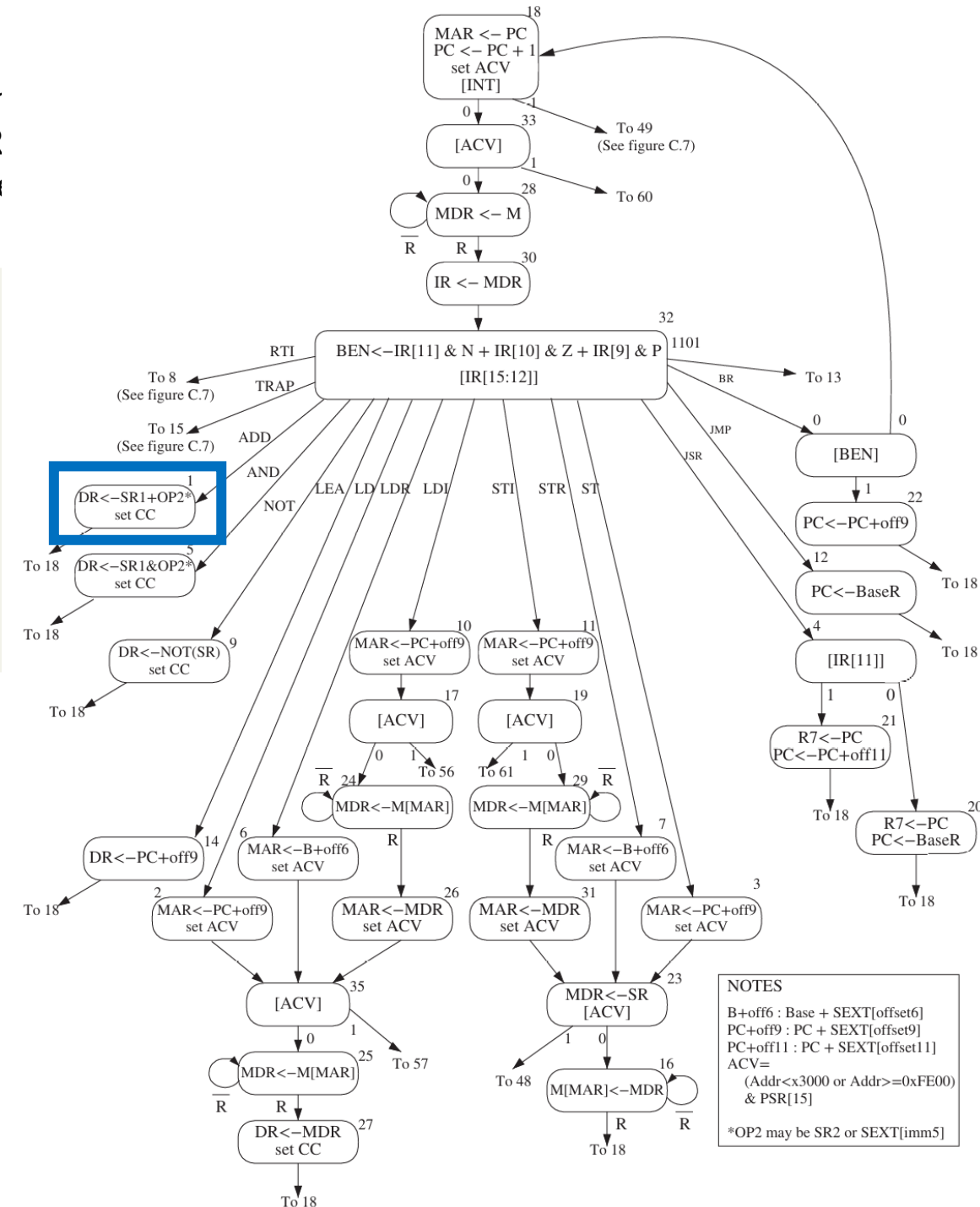
LD R1, A

LD R1, B

ADD R2, R2, #1

ADD R0, R1, R0

Cycle Number	State Number	Information			
(d) 11	(e) 27	LD.REG: 1	DRMUX: 0 (f)	GateMDR: 1 (g)	
		LD.CC: 1 (h)	GateALU: 0 (i)	GatePC: 0 (j)	
16	30	LD.MDR: 0 (k)	MDR: (m) x2209	IR: (n) x2009	
		LD.IR: 1 (l)			
50	(o) 1	LD.REG: 1	MDR: (p) x14A1	DRMUX: 2 (q)	
		BUS: x0001		GateMDR: 0 (r)	
57	1	PC: (s)	IR: (t) x1040	GateALU: 1 (u)	
		BUS: x0003		GatePC: 0 (v)	
(w)	22	ADDR1MUX: (x)	ADDR2MUX: (y)		
		LD.PC: 1	PC: x3008	PCMUX: ADDER	



(1) Insert the missing instructions in the program and the missing information in the table. Cycle numbering starts at 1. That is, cycle 1 is the first clock cycle of the processing of LD R0, A. Note that we are asking for the value of the registers DURING each clock cycle. We have not said anything about the number of clock cycles a memory access takes, but you do have enough information to figure that out for yourself.

```

15
14 C      .BLKW #1
13 B      .FILL ----- (c)
12 A      .FILL #5
11
10 DONE   ST R2, C
9         BRnzp AGAIN
8         ----- (b) ADD R0, R1, R0
7 AGAIN   ----- (a) ADD R2, R2, #1
6         AND R2, R2, #0
5         7 ADD R1, R1, #1
4         NOT R1, R1
3         LD R1, B
2         11 LD R0, A
1         .ORIG x3000

```

LD R1, A

LD R1, B

ADD R2, R2, #1

ADD R0, R1, R0

Cycle Number	State Number	Information		
(d) 11	(e) 27	LD.REG: 1	DRMUX: 0 (f)	GateMDR: 1 (g)
		LD.CC: 1 (h)	GateALU: 0 (i)	GatePC: 0 (j)
16	30	LD.MDR: 0 (k)	MDR: (m) x2209	IR: (n) x2009
		LD.IR: 1 (l)		
50	(o)1	LD.REG: 1	MDR: (p) x14A1	DRMUX: 2 (q)
		BUS: x0001		GateMDR: 0 (r)
57	1	PC: (s)	IR: (t) x1040	GateALU: 1 (u)
		BUS: x0003		GatePC: 0 (v)
(w)	22	ADDR1MUX: (x)	ADDR2MUX: (y)	
		LD.PC: 1	PC: x3008	PCMUX: ADDER

(1) Insert the missing instructions in the program and the missir
That is, cycle 1 is the first clock cycle of the processing of LD R
DURING each clock cycle. We have not said anything about the
do have enough information to figure that out for yourself.

```

.Orig x3000
11 LD R0, A
LD R1, B
NOT R1, R1
7 ADD R1, R1, #1
AND R2, R2, #0
AGAIN ----- (a) ADD R2, R2, #1
----- (b) ADD R0, R1, R0
BRnzp AGAIN
DONE ST R2, C
HALT
A .FILL #5
B .FILL ----- (c)
C .BLKW #1
.END

```

LD R1, A

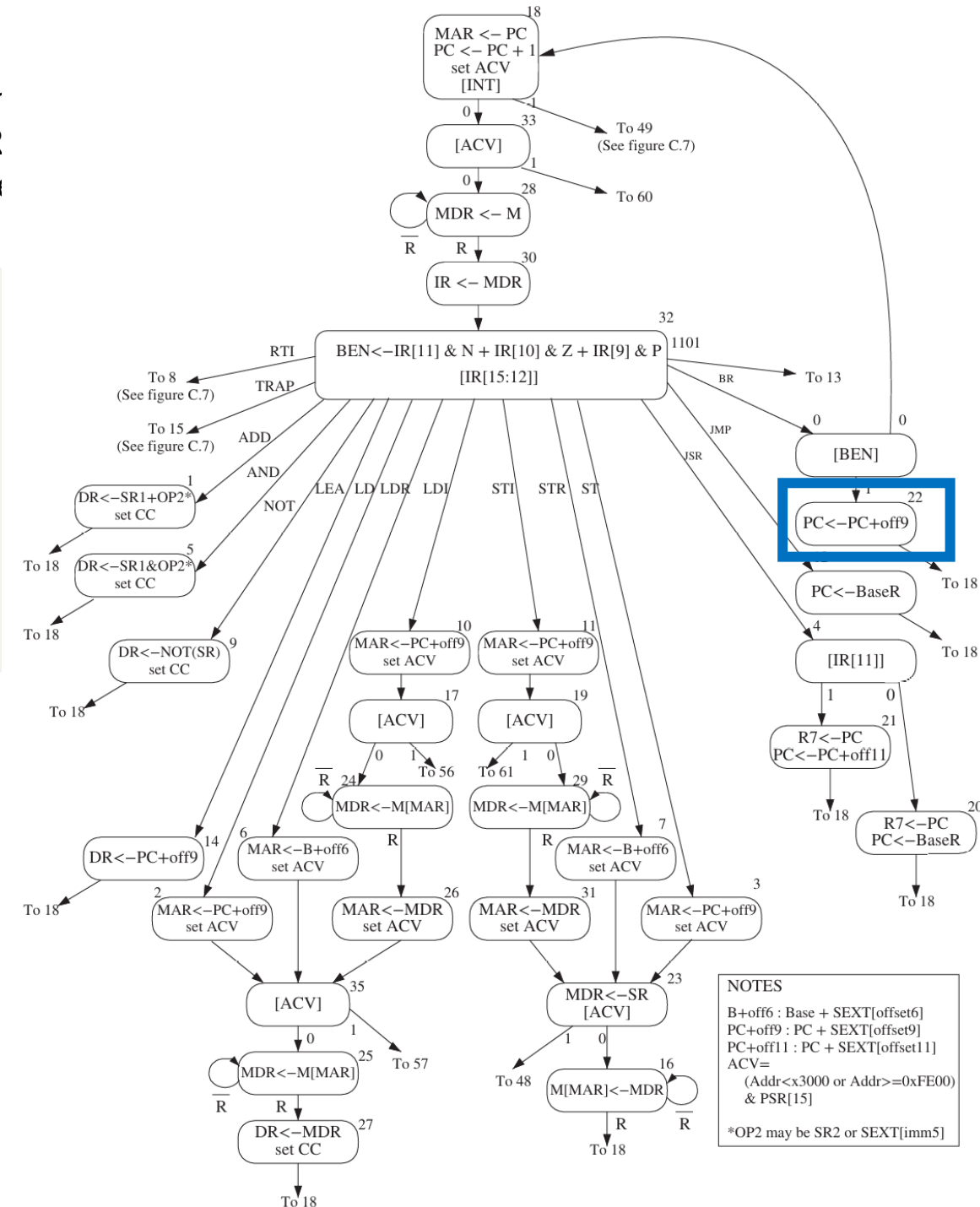
LD R1, B

ADD R2, R2, #1

ADD R0, R1, R0

BRnzp AGAIN

Cycle Number	State Number	Information			
(d) 11	(e) 27	LD.REG: 1	DRMUX: 0 (f)	GateMDR: 1 (g)	
		LD.CC: 1 (h)	GateALU: 0 (i)	GatePC: 0 (j)	
16	30	LD.MDR: 0 (k)	MDR: (m) x2209	IR: (n) x2009	
		LD.IR: 1 (l)			
50	(o) 1	LD.REG: 1	MDR: (p) x14A1	DRMUX: 2 (q)	
		BUS: x0001		GateMDR: 0 (r)	
57	1	PC: (s)	IR: (t) x1040	GateALU: 1 (u)	
		BUS: x0003		GatePC: 0 (v)	
(w) 65	22	ADDR1MUX: PC (x)	ADDR2MUX: (y) PC Offset9		
		LD.PC: 1	PC: x3008	PCMUX: ADDER	



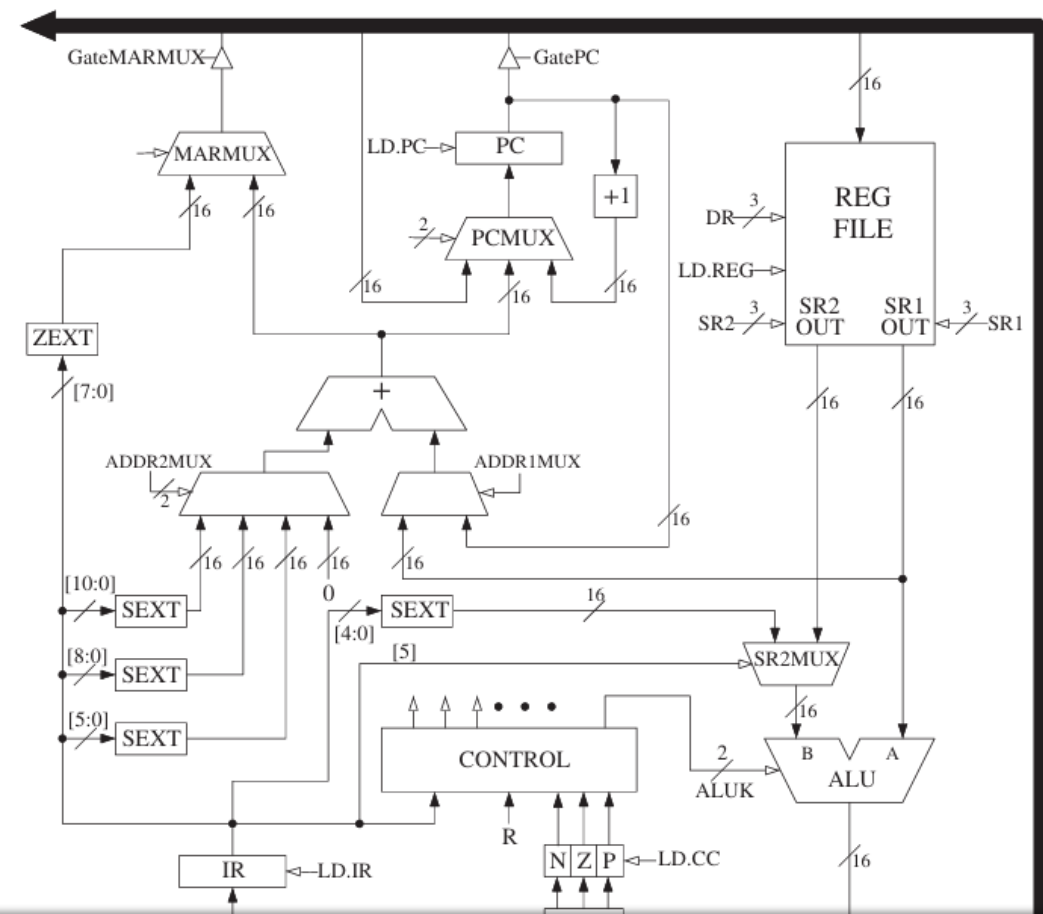
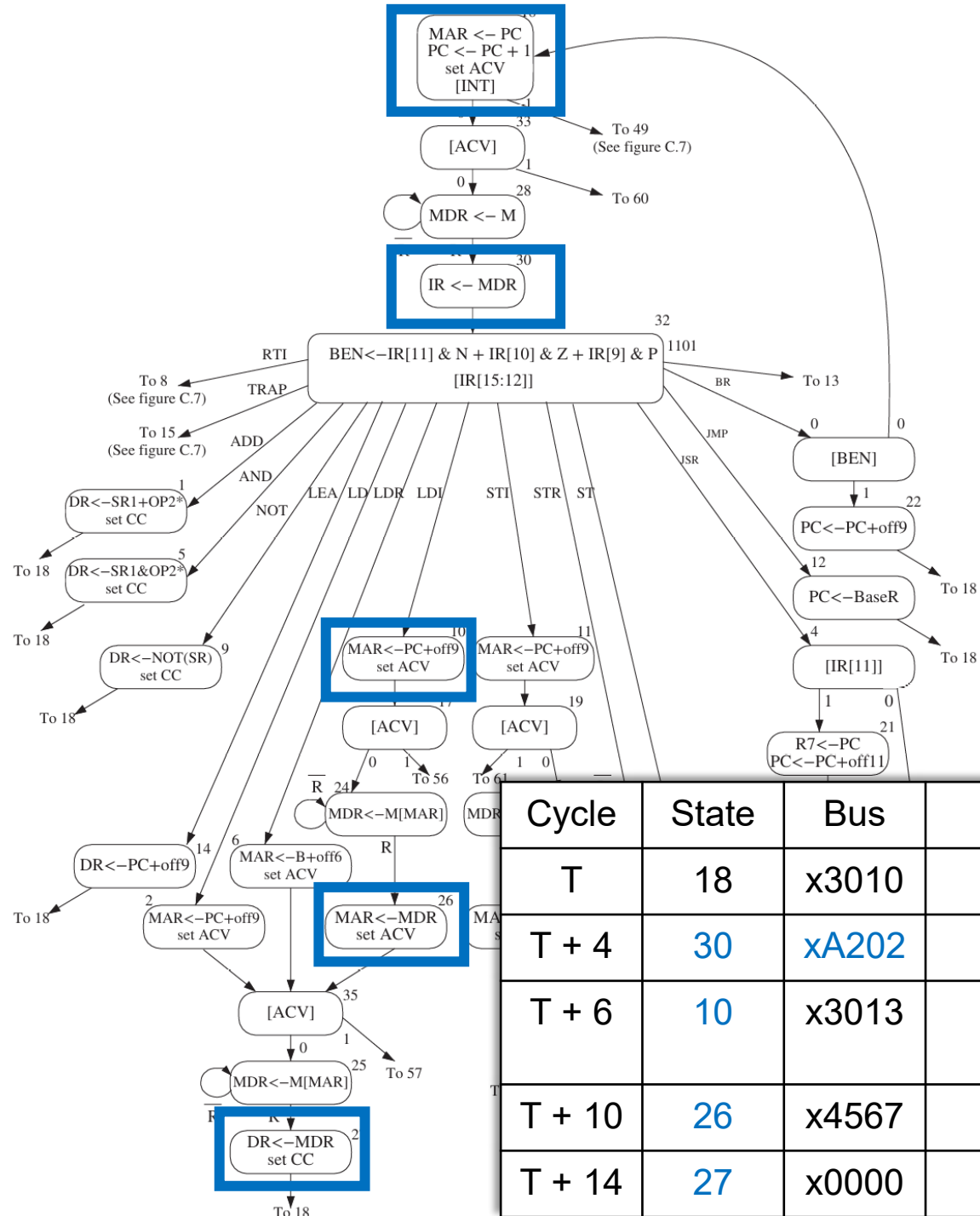
During the execution of an LC-3 program, an instruction in the program starts executing at clock cycle T and requires 15 cycles to complete. The table lists ALL five clock cycles during the processing of this instruction, which require use of the bus. The table shows for each of those clock cycles: which clock cycle, the state of the state machine, the value on the bus, and the important control signals that are active during that clock cycle.

Cycle	State	Bus	Important Control Signals for This Cycle
T	18	x3010	LD.MAR = 1, LD.PC = 1, PCMux = PC+1, GatePC = 1
T + 4			
T + 6		x3013	
T + 10		x4567	
T + 14		x0000	LD.REG = 1, LD.CC = 1, GateMDR = 1, DR = 001

During the execution of an LC-3 program, an instruction in the program starts executing at clock cycle T and requires 15 cycles to complete. The table lists ALL five clock cycles during the processing of this instruction, which require use of the bus. The table shows for each of those clock cycles: which clock cycle, the state of the state machine, the value on the bus, and the important control signals that are active during that clock cycle.

Cycle	State	Bus	Important Control Signals for This Cycle
T	18	x3010	LD.MAR = 1, LD.PC = 1, PCMux = PC+1, GatePC = 1
T + 4			
T + 6		x3013	
T + 10		x4567	
T + 14		x0000	LD.REG = 1, LD.CC = 1, GateMDR = 1, DR = 001

最后一个状态要求将 MDR 放到总线，说明是内存读取操作。
结合状态机和数据通路查看访问总线的状态有哪些。



Cycle	State	Bus	Important Control Signals for This Cycle
T	18	x3010	LD.MAR = 1, LD.PC = 1, PCMux = PC+1, GatePC = 1
T + 4	30	xA202	Gate.MDR = 1, LD.IR = 1
T + 6	10	x3013	ADDR1MUX = PC, ADDR2MUX = SEXT(IR[8:0]), MARMUX = ADDR.ADD, Gate.MARMUX = 1
T + 10	26	x4567	
T + 14	27	x0000	LD.REG = 1, LD.CC = 1, GateMDR = 1, DR = 001

The following shows a fragment of the HALT service routine. Under normal circumstances, will the instruction at address x036C be executed? If not, explain the purpose of this instruction.

Address	Instruction
x0366	<i>LEA R0, TRAP_HALT_MSG</i>
x0367	<i>PUTS</i>
x0368	<i>LDI R0, OS_MCR</i>
x0369	<i>LD R1, MASK_HI</i>
x036A	<i>AND R0, R0, R1</i>
x036B	<i>STI R0, OS_MCR</i>
x036C	<i>BRnzp TRAP_HALT</i>

在 x036B 处，Machine control register (MCR) 的最高位被清空，时钟被停止，因此 x036C 不会被执行。

地址 x036C 处的 BRnzp 指令实际上是一个跳转到 TRAP SR 入口处的指令，它的目的是确保即使计算机出现异常没能成功停止时钟时也能防止意外行为。

The following shows a fragment of the HALT service routine. Under normal circumstances, will the instruction at address x036C be executed? If not, explain the purpose of this instruction.

Address	Instruction
x0366	<i>LEA R0, TRAP_HALT_MSG</i>
x0367	<i>PUTS</i>
x0368	<i>LDI R0, OS_MCR</i>
x0369	<i>LD R1, MASK_HI</i>
x036A	<i>AND R0, R0, R1</i>
x036B	<i>STI R0, OS_MCR</i>
x036C	<i>BRnzp TRAP_HALT</i>

在 x036B 处，Machine control register (MCR) 的最高位被清空，时钟被停止，因此 x036C 不会被执行。

地址 x036C 处的 BRnzp 指令实际上是一个跳转到 TRAP SR 入口处的指令，它的目的是确保即使计算机出现异常没能成功停止时钟时也能防止意外行为。

Consider the following two C code snippets:

1. What will be the output of each code snippet?
2. These two code snippets demonstrate two different ways of copying arrays. What are their respective advantages and disadvantages?
3. In code snippet (a), after `b = a`, can `b` point to a different array? Can `a` be reassigned (e.g., `a = b`)?

```
int a[]={0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
int *b = a;
b[5] = 42;
printf("%d %d", a[5], b[5]);
```

```
int a[]={0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
int *b = malloc(10 * sizeof(int));
for (int i = 0; i < 10; i++) {
    b[i] = a[i];
}
b[5] = 42;
printf("%d %d", a[5], b[5]);
```

(C 转 LC-3)

Consider the following C code:

```
int arr[10];
```

```
int *parr = &arr[0];
```

Assume the size of an int variable is one word and the value of variable parr is stored in register R0.

- `*parr = *parr - 1;`

```
LDR R1, R0, #0
```

```
ADD R1, R1, #-1
```

```
STR R1, R0, #0
```